学　　号：2022217559　　密　　级：公开

合肥工业大学

Hefei University of Technology

本科毕业设计（论文）

UNDERGRADUATE THESIS

类　　型：	毕业设计
题　　目：	基于整数规划的房屋建模系统设计与实现
专业名称：	物联网工程
入校年份：	2022 级
学生姓名：	王景涛
指导教师：	吴文明　副教授
学院名称：	计算机与信息学院
完成时间：	2026 年 5 月

合　肥　工　业　大　学

本科毕业设计（论文）

基于整数规划的房屋建模系统设计与实现

学生姓名：王景涛

学生学号：2022217559

指导教师：吴文明　副教授

专业名称：物联网工程

学院名称：计算机与信息学院

2026 年 5 月

A Dissertation Submitted for the Degree of Bachelor

Automated Residential Floor Plan Generation System Based on Integer Programming

By

Wang Jingtao

Hefei University of Technology

Hefei, Anhui, P.R.China

May, 2026

毕业设计（论文）独创性声明

本人郑重声明：所呈交的毕业设计（论文）是本人在指导教师指导下进行独立研究工作所取得的成果。据我所知，除了文中特别加以标注和致谢的内容外，设计(论文)中不包含其他人已经发表或撰写过的研究成果，也不包含为获得 合肥工业大学 或其他教育机构的学位或证书而使用过的材料。对本文成果做出贡献的个人和集体，本人已在设计（论文）中作了明确的说明，并表示谢意。

毕业设计（论文）中表达的观点纯属作者本人观点，与合肥工业大学无关。

毕业设计（论文）作者签名：　　签名日期：2026 年 5 月 15 日

毕业设计（论文）版权使用授权书

本学位论文作者完全了解 合肥工业大学 有关保留、使用毕业设计（论文）的规定，即：除保密期内的涉密设计（论文）外，学校有权保存并向国家有关部门或机构送交设计（论文）的复印件和电子光盘，允许设计（论文）被查阅或借阅。本人授权 合肥工业大学 可以将本毕业设计（论文）的全部或部分内容编入有关数据库，允许采用影印、缩印或扫描等复制手段保存、汇编毕业设计（论文）。

(保密的毕业设计（论文）在解密后适用本授权书)

学位论文作者签名：　　指导教师签名：

签名日期：2026 年 5 月 15 日　　签名日期：2026 年 5 月 15 日

摘　要

住宅户型设计是建筑设计中最为关键的基础环节之一，直接影响居住者的生活品质和建筑的空间利用效率。传统的户型设计过程依赖于建筑师的经验和手动绘图，设计周期长、效率低下，难以在短时间内产生多样化的备选方案。针对上述问题，本文设计并实现了一个基于整数规划（Integer Programming, IP）的住宅户型自动生成系统。

该系统以数学优化为核心，将户型布局问题建模为一个约束满足与目标优化的统一数学框架。具体而言，系统将建筑轮廓离散化为均匀网格，每个房间表示为一个轴对齐矩形，其位置和尺寸由整数变量表示，房间的朝向选择由二元变量控制。系统综合建模了房间尺寸约束、宽高比约束、边界约束、不重叠约束、邻接约束、扩展邻接约束和靠墙约束等七类建筑规则，将其转化为线性等式或不等式约束。在此基础上，构造了包含面积惩罚项、尺寸偏差惩罚项和辅助面积惩罚项的目标函数，形成一个标准的整数规划模型，并调用 Gurobi 商业优化求解器进行精确求解。

在算法实现的基础上，本文开发了一个功能完善的交互式软件系统。系统采用 Python 语言实现，以 Tkinter 构建图形用户界面，主要功能模块包括：可视化约束编辑器（支持房间尺寸表格编辑、邻接关系图编辑、扩展邻接编辑等）、核心优化引擎（调用 Gurobi 求解整数规划模型）、户型渲染器（生成带尺寸标注、门窗标识、家具布局的平面图）、日照采光模拟器（基于太阳轨迹天文学模型计算各房间日照时数）、装修预算估算器（支持经济简装、标准精装、高端豪装三档预算评估）、腾讯混元 3D 大模型集成（文生 3D 与图生 3D 自动建模）以及智能户型推荐引擎（基于用户需求的多维度匹配推荐）。

系统测试表明，针对 5 至 15 个房间的典型住宅户型，系统能够在数秒至数十秒内生成满足全部设计约束的优化布局方案；日照模拟功能能够准确反映不同朝向和季节下的采光差异；预算估算功能为用户提供了直观的成本参考。本系统的设计与实现为住宅户型自动化设计提供了一种高效、可靠的解决方案，具有较强的工程实用价值。

关键词：住宅户型生成；整数规划；布局优化；三维建模

ABSTRACT

Residential floor plan design is one of the most fundamental aspects of architectural design, directly affecting occupants' quality of life and spatial utilization efficiency. Traditional floor plan design relies on architects' experience and manual drafting, resulting in long design cycles and low efficiency, making it difficult to generate diverse alternatives in a short time. To address these issues, this thesis designs and implements an automated residential floor plan generation system based on Integer Programming (IP).

The system centers on mathematical optimization, modeling the floor plan layout problem as a unified mathematical framework combining constraint satisfaction and objective optimization. Specifically, the system discretizes the building outline into a uniform grid, represents each room as an axis-aligned rectangle whose position and dimensions are expressed as integer variables, and controls room orientation selection through binary variables. The system comprehensively models seven categories of architectural rules, including room dimension constraints, aspect ratio constraints, boundary constraints, non-overlap constraints, adjacency constraints, extended adjacency constraints, and wall-attachment constraints, transforming them into linear equality or inequality constraints. On this basis, an objective function incorporating area penalty terms, dimension deviation penalty terms, and auxiliary area penalty terms is constructed, forming a standard IP model solved using the Gurobi commercial optimization solver.

Based on the algorithm implementation, this thesis develops a fully functional interactive software system. The system is implemented in Python with a Tkinter-based graphical user interface, featuring the following main functional modules: a visual constraint editor (supporting room dimension table editing, adjacency graph editing, and extended adjacency editing), a core optimization engine (invoking Gurobi to solve the IP model), a floor plan renderer (generating plans with dimension annotations, door/window markings, and furniture layout), a daylight simulation module (computing room-level sunshine duration based on solar trajectory astronomical models), a renovation budget estimator (supporting three budget tiers: economy, standard, and luxury), Tencent Hunyuan 3D model integration (text-

to-3D and image-to-3D automatic modeling), and an intelligent floor plan recommendation engine (multi-dimensional matching recommendation based on user requirements).

System testing demonstrates that for typical residential layouts with 5 to 15 rooms, the system can generate optimized layouts satisfying all design constraints within seconds to tens of seconds. The daylight simulation function accurately reflects differences in lighting conditions across orientations and seasons. The budget estimation function provides users with intuitive cost references. The design and implementation of this system offer an efficient and reliable solution for automated residential floor plan design, with strong practical engineering value.

KEYWORDS: Residential Floor Plan Generation; Integer Programming; Layout Optimization; 3D Modeling

目　录

1 绪论 1
1.1 研究背景与意义 1
1.2 国内外研究现状 1
1.3 本文主要工作 3
1.4 论文组织结构 5
2 相关理论与技术基础 6
2.1 整数规划概述 6
2.2 Gurobi 优化求解器 6
2.3 大 M 法条件约束建模 7
2.4 系统开发技术栈 7
2.5 本章小结 10
3 系统需求分析与总体设计 11
3.1 系统功能需求分析 11
3.2 系统非功能需求 12
3.3 系统总体架构 12
3.4 系统模块划分 13
3.5 系统界面布局设计 14
3.6 本章小结 14
4 基于整数规划的户型布局优化算法 15
4.1 问题形式化定义 15
4.2 变量定义与初始化 16
4.3 约束建模 16
4.3.1 尺寸约束与朝向控制 16
4.3.2 不重叠约束 17

4.3.3 邻接约束 17
4.3.4 边界约束 18
4.3.5 扩展邻接约束 18
4.3.6 靠墙约束 18
4.3.7 其他约束 18
4.4 目标函数构造 19
4.5 求解流程 19
4.6 更新模式 20
4.7 算法扩展与改进 21
4.8 三维建模数据转换与生成策略 22
4.9 本章小结 25
5 系统详细设计与实现 26
5.1 约束编辑器设计与实现 26
5.2 核心优化引擎实现 29
5.3 户型渲染器实现 31
5.4 日照采光模拟器实现 32
5.5 装修预算估算器实现 32
5.6 智能推荐引擎实现 34
5.7 3D 建模集成实现 36
5.8 本章小结 39
6 系统测试与结果分析 40
6.1 测试环境与配置 40
6.2 优化算法正确性验证 40
6.3 多房间规模性能测试 41
6.4 户型评估功能验证 42
6.5 日照模拟功能验证 43
6.6 预算估算功能验证 44

6.7 本章小结 44
7 总结与展望 46
7.1 工作总结 46
7.2 系统存在的不足 46
7.3 未来工作展望 47
7.4 本章小结 48
参考文献 49
致谢 51
附录 52

插图清单

图 1.1 住宅户型设计背景示意图 2
图 1.2 系统整体技术框架图 4
图 2.1 现有三维建模方法：利用建模软件 8
图 2.2 现有三维建模方法：IFC4 建筑信息模型 8
图 2.3 混元 3D 大模型生成原理示意图 9
图 4.1 二维户型结果到三维建模输入的转换流程 22
图 4.2 文图混合三维生成模式示意图 24
图 5.1 系统主界面 26
图 5.2 项目管理界面 27
图 5.3 约束编辑器窗口 27
图 5.4 约束文件编辑器二级界面 28
图 5.5 读取边界文件后的界面效果 29
图 5.6 整数规划优化生成的户型布局 30
图 5.7 求解过程监控历史 30
图 5.8 带标注的户型平面图 31
图 5.9 日照采光模拟界面 33
图 5.10 装修预算估算界面 33
图 5.11 装修预算估算参考界面 34
图 5.12 户型智能推荐引擎界面 36
图 5.13 户型智能推荐结果界面 36
图 5.14 3D 建模功能界面 37
图 5.15 手动生成 3D 模型界面 38
图 5.16 腾讯混元 3D 模型生成结果 39
图 6.1 系统运行结果界面 41
图 6.2 户型评估报告界面 43

表格清单

表 3.1 系统功能需求分析表..... 12
表 5.1 智能推荐引擎评分维度 35
表 6.1 5 房间测试用例约束配置 40
表 6.2 5 房间测试用例生成结果 41
表 6.3 不同房间数量下的求解性能对比..... 41

1 绪论

1.1 研究背景与意义

住宅户型设计是建筑设计流程中的重要基础环节，其方案质量直接影响空间利用效率、居住舒适度和后续施工表达[1]。传统户型设计主要依赖建筑师经验和多轮人工修改，设计周期较长，且难以在短时间内系统比较大量备选方案[2]。与此同时，房间尺寸、宽高比、邻接关系、边界贴合和走廊宽度等设计规则具有较强的结构化特征，能够通过数学语言进行明确描述。因此，将住宅户型设计转化为可计算、可约束和可优化的问题，具有较高的研究价值和工程应用意义。

计算机辅助户型生成并非全新的研究方向。从早期计算机辅助绘图到参数化设计和生成式设计[3]，计算机辅助建筑设计技术已经形成了较为丰富的研究基础[4]。Flager 等人[5] 将建筑行业优化流程与航空航天行业设计流程进行比较，指出建筑设计同样包含复杂的多目标权衡与约束协调问题。在多种技术路线中，将户型布局建模为数学优化问题并利用成熟求解器进行求解，是一种可解释性较强且工程实现路径清晰的方法。该方法能够明确说明各类约束的满足情况和目标函数的优化方向，相比完全依赖数据驱动的黑箱模型，更便于调试、验证和工程落地。

本文所实现系统的核心思路是将住宅户型设计问题转化为整数规划模型。系统将房间尺寸、宽高比、邻接关系和边界贴合等建筑设计规则转化为等式或不等式约束，并将面积利用率、尺寸偏差和辅助空间面积等因素纳入目标函数。模型建立后，由 Gurobi 优化求解器完成求解。为增强系统的实用性，本文还集成了可视化渲染、日照分析、装修预算估算和三维建模等辅助功能，形成如图 1.1所示的完整流程。

本文工作的意义主要体现在三个方面。理论层面，本文围绕住宅户型布局建立了较为完整的整数规划约束体系和目标函数设计方法；技术层面，本文将算法模型实现为可交互的软件系统，并形成从约束输入到结果展示的完整流程；应用层面，系统能够为建筑师或普通用户快速提供满足基本约束的户型初始方案，为后续人工深化设计提供参考。

1.2 国内外研究现状

户型自动生成属于建筑学、计算机科学和运筹优化等学科的交叉研究方向。现有方法大致可以分为形状语法方法、随机优化方法、机器学习方法和数学规划方法

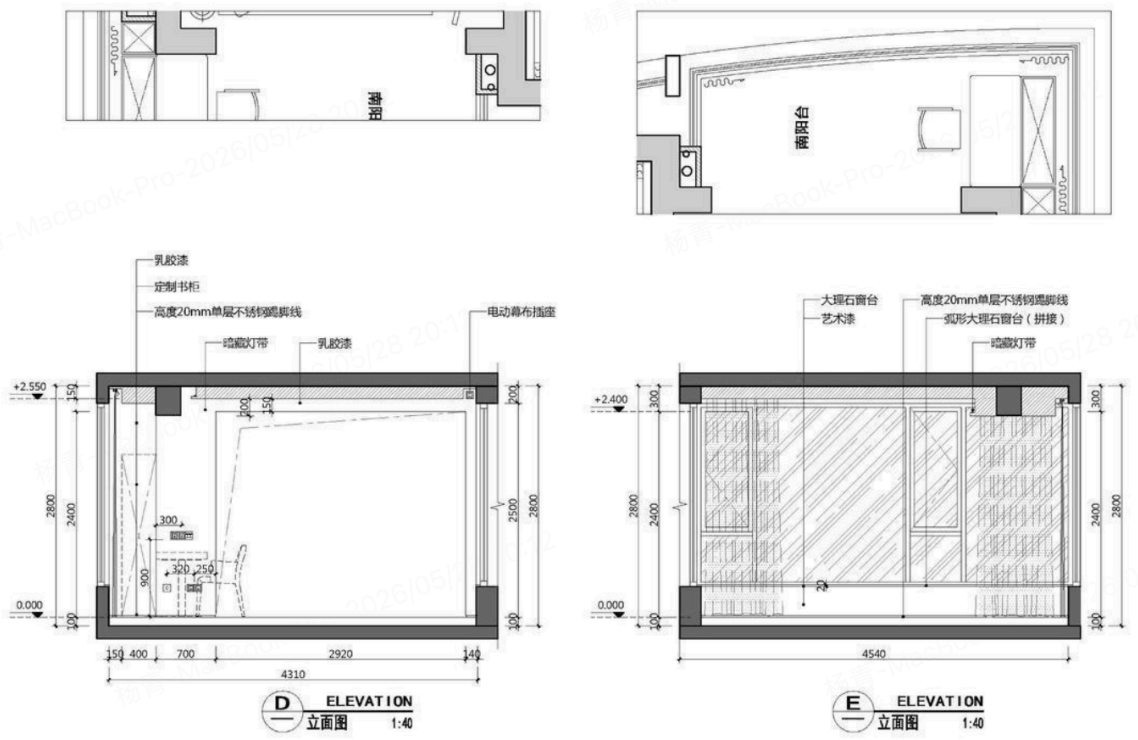


图 1.1　住宅户型设计背景示意图

四类。

形状语法方法。形状语法是较早用于建筑形态生成的研究方法。Stiny 等人[6] 提出通过定义形状变换规则生成设计结果。随后，Parish 和 Müller[7] 将程序化建模应用于城市生成，Müller 等人[8] 进一步将程序化规则扩展到建筑立面和建筑形体建模，表明规则驱动方法在建筑几何生成中具有较强表达能力。然而，该类方法高度依赖人工规则设计，生成结果的多样性和合理性受规则库质量影响较大。

随机优化方法。随机优化方法是一类在搜索过程中引入随机性来探索复杂解空间、避免局部最优的启发式算法。Li 等人[9] 通过物理启发参数模型与进化优化相结合，生成多样化、高质量的设计解决方案，该方法集成了进化优化元启发式算法进行自动化空间布局规划。李若天[10] 基于遗传算法，通过提取和转译适应性住宅的套内设计规则，提出了一种能够快速生成灵活可变平面布局的方法。李冰瑶[11] 将住宅规划布局的多性能目标优化转化为多变量 - 多目标问题，并采用多目标优化算法与

集成优化方法，实现了住区布局的智能生成与性能提升。Laignel 等人[12] 将约束规划与遗传优化结合用于公寓布局生成。这类方法能够处理复杂非线性约束，但求解结果受初始条件、参数配置和搜索时间影响较大，难以稳定保证最优性。

机器学习方法。近年来，GAN、VAE 和扩散模型等生成式模型逐渐被用于户型生成任务。Nauata 等人[13] 提出的 House-GAN 利用图神经网络和 GAN 生成户型图；Hu 等人[14] 提出 Graph2Plan，从布局图直接学习生成户型平面图；Liu 等人[15] 从平面图图像矢量化角度研究墙体和房间结构恢复，为平面图数据处理提供了参考。相关研究还借鉴压缩感知和稀疏表示思想，从有限观测中恢复结构化信息[16]。这类方法具有较强的生成能力，但模型可解释性相对不足，且对高质量训练数据依赖较强，生成结果也可能出现约束不满足或空间逻辑不合理的问题。

数学规划方法。通过制定几何布局的数学问题与一组变量、目标和约束，优化算法可用于解决布局设计的问题[17]，并利用优化求解器进行求解。Wu 等人[18] 提出分层布局优化框架，从粗到细生成室内布局，并利用整数规划思想同时优化房间位置和尺寸。Haratyunyan[19] 也开展了相关研究。该类方法的优势在于约束表达清晰、求解结果可验证，并能够在一定规模下获得具有数学保证的可行解或高质量解。本文在上述研究基础上，结合住宅户型设计场景进行了工程化实现与功能扩展。

1.3 本文主要工作

围绕基于整数规划的房屋建模系统设计与实现，本文主要开展了以下工作。系统整体技术框架如图 1.2所示。

（1）构建了面向住宅户型布局的整数规划模型。本文将房间的位置、尺寸等空间布局要素表示为整数决策变量，将房间朝向表示为二元决策变量，并对尺寸约束、不重叠约束、邻接约束、边界约束、扩展邻接约束和靠墙约束等建筑设计规则进行数学建模。在此基础上，构造包含面积利用、尺寸偏差和辅助面积惩罚等因素的目标函数，从而形成可由优化求解器处理的整数规划模型。

（2）设计并实现了可视化约束编辑模块。为降低整数规划模型的输入复杂度，系统提供了房间尺寸表格编辑、邻接关系图形化编辑、扩展邻接关系配置以及靠边约束设置等交互功能，使用户能够以图形化方式完成户型设计约束的录入、修改和校验。

（3）实现了户型渲染与方案评估功能。系统将优化求解得到的坐标和尺寸数据

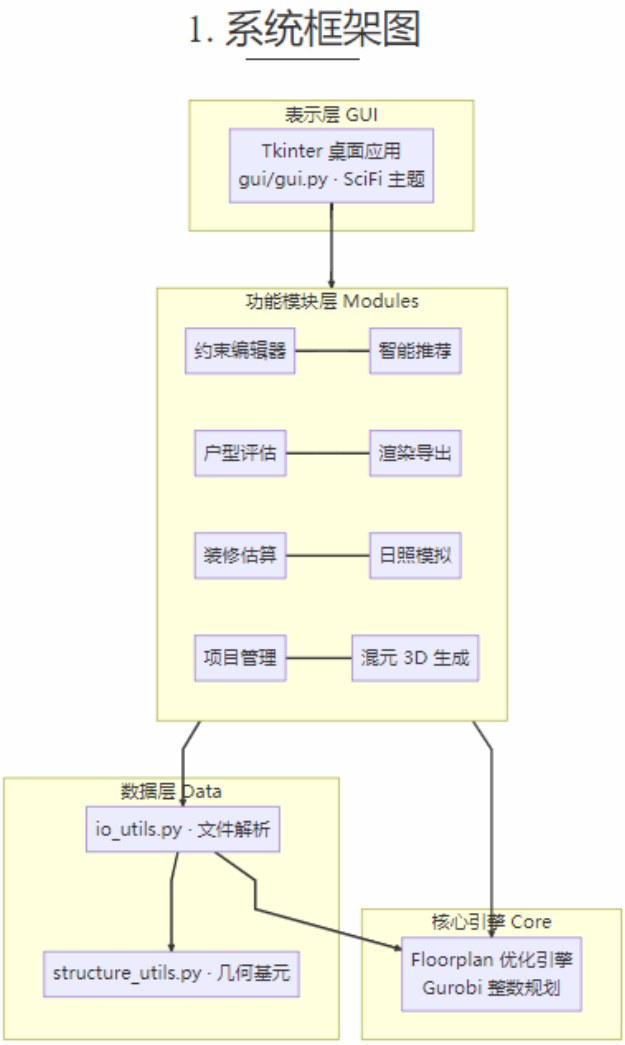


图 1.2　系统整体技术框架图

转换为可视化户型平面图，并在图中展示房间色块、尺寸标注、门窗位置、家具布置和指北针等信息。同时，系统从空间利用率、自然采光、动线效率、动静分区和邻接满意度等维度对生成方案进行综合评价，为用户选择和改进方案提供量化依据。

（4）集成了多种辅助分析与建模模块。系统实现了基于太阳轨迹模型的日照采光模拟功能、面向不同装修档次的预算估算功能、基于用户需求与模板库匹配的智能推荐功能，并集成腾讯混元 3D 大模型接口实现三维模型生成，从而扩展了系统在方案分析、成本估算和空间展示方面的应用能力。

1.4 论文组织结构

论文共七章，各章内容安排如下：

第 1 章为绪论，主要介绍研究背景与意义、国内外研究现状以及本文主要工作。

第 2 章介绍相关理论与技术基础，包括整数规划、Gurobi 优化求解器、大 M 法条件约束建模以及系统开发技术栈。

第 3 章进行系统需求分析与总体设计，说明系统功能需求、模块划分、总体架构和界面设计思路。

第 4 章阐述基于整数规划的户型布局优化算法，包括问题形式化、变量定义、约束建模、目标函数构造、求解流程和三维建模数据转换策略。

第 5 章介绍系统详细设计与实现，重点说明约束编辑器、优化引擎、渲染器、日照模拟器、预算估算器和三维建模等模块的实现方法。

第 6 章开展系统测试与结果分析，通过不同规模的用例验证算法正确性、求解性能和功能完整性。

第 7 章总结全文工作，分析系统存在的不足，并提出后续改进方向。

2 相关理论与技术基础

2.1 整数规划概述

整数规划（Integer Programming, IP）是在给定约束条件下优化目标函数，并要求部分或全部决策变量取整数值的一类数学规划方法。户型布局中的房间位置、尺寸和朝向选择均具有明显的离散特征，因此适合采用整数规划进行建模。其标准形式可表示如下：

$$\begin{aligned} \min_{x,y} \quad & \frac{1}{2}x^T Q x + c^T x + d^T y \\ \text{s.t.} \quad & Ax + By \le b \\ & x \in \mathbb{R}^n,\ y \in \mathbb{Z}^m \end{aligned} \tag{2.1}$$

x 为连续变量，y 为整数变量，Q 通常要求为半正定矩阵，以保证目标函数具有较好的求解性质。A 和 B 为约束系数矩阵，b 为约束右端项，c 和 d 为目标函数系数。

当 y 中的变量限定为 0 或 1 时，模型可以表达逻辑条件、选择关系和开关行为等离散决策。本系统大量使用二元变量控制房间朝向，并配合大 M 法实现条件约束的激活与松弛。

整数规划问题通常属于 NP-hard 问题[20]，随着变量规模增加，求解复杂度会显著上升。现代商业求解器（如 Gurobi、CPLEX）综合使用分支定界、割平面、启发式搜索和预处理等技术[21]，使中等规模问题能够在可接受时间内获得高质量解。

2.2 Gurobi 优化求解器

Gurobi[22] 是当前应用较广的数学规划求解器之一，能够处理 LP、MILP、QP、整数规划和二次约束规划等多类问题。其优势主要体现在高效的分支定界实现、较强的预处理能力以及良好的并行计算性能。

在 Python 环境中使用 Gurobi 时，通常先创建 `Model` 对象作为模型容器，再通过 `addVar` 创建决策变量，通过 `addConstr` 添加约束，通过 `setObjective` 设置目标函数，最后调用 `optimize` 执行求解。求解结束后，可通过 `Status` 判断求解状态，通过 `ObjVal` 读取目标函数值，并通过变量的 `X` 属性获取最优解取值。

本系统主要使用了 Gurobi 的三项能力：（1）二次目标函数，用于直接描述面积利用和尺寸偏差等目标；（2）参数调优，通过 `MIPFocus=2`、`Heuristics=0.1` 和

`TimeLimit` 等参数平衡求解质量与求解时间；（3）Callback 回调机制，用于在求解过程中实时监控进度并在满足条件时提前终止求解。

2.3 大 M 法条件约束建模

大 M 法（Big-M Method）是整数规划中表达逻辑条件的常用技术。其基本思想是引入足够大的常数 M（本系统中记为 LARGE，取值为 200），并配合二元变量构造条件约束，从而实现“若某条件成立，则某约束生效”的建模效果。

例如，若需要表达“当二元变量 $z = 0$ 时，$f(x) \leq 0$ 必须成立”，可使用大 M 法写成如下不等式：

$$f(x) \leq M \cdot z$$

当 $z = 0$ 时，该约束被强制执行；当 $z = 1$ 时，由于 M 足够大，约束被有效松弛。该方法形式简洁，且在工程建模中具有较强实用性。

在户型布局问题中，大 M 法最典型的应用是不重叠约束。对于任意两个矩形房间 i 和 j，二者至少需要在上、下、左、右中的一个方向上相互分离。系统引入四个二元变量 $\delta_0, \delta_1, \delta_2, \delta_3$，通过约束 $\delta_0 + \delta_1 + \delta_2 + \delta_3 \geq 1$ 保证至少一个分离关系成立，并分别为四个方向建立对应的大 M 约束。

2.4 系统开发技术栈

系统采用 Python 语言实现。技术栈选择以轻量、稳定和易部署为主要原则，具体组成如下：

GUI 框架：系统选用 Python 标准库中的 Tkinter 作为图形界面框架。该框架轻量、跨平台且无需额外安装依赖。系统在 Tkinter 原生控件基础上自定义了 SciFiButton 等暗色主题组件，并采用深蓝色背景与青色强调色构建统一界面风格。

科学计算：NumPy 用于矩阵和数组运算，主要服务于日照模拟中的光照强度累积。Matplotlib 用于绘制户型平面图、日照热力图、预算柱状图和饼图，能够满足系统可视化输出需求。

优化求解：系统通过 Gurobi Python API（gurobipy 包）调用优化求解器，并使用其学术许可完成算法求解与实验验证。

3D 模型生成：三维建模是本系统区别于普通二维户型生成工具的重要功能之一。传统三维建模方法通常包括人工建模、BIM/IFC 数据驱动建模、程序化几何建

模以及近年来兴起的三维生成大模型方法。人工建模主要依赖 SketchUp、3ds Max、Blender 等建模软件，由设计人员根据平面图手动建立墙体、门窗、家具和材质模型，几何表达较为灵活，但建模周期长、专业门槛高，不适合自动化批量生成场景。BIM/IFC 方法则以建筑信息模型为核心，通过墙体、楼板、门窗、构件族和空间对象等结构化实体描述建筑空间，具有较强的工程表达能力和协同设计价值，但需要较完整的建筑构件语义和规范化数据输入。程序化建模方法通过规则、脚本或参数化模板生成三维几何体，适合将二维户型坐标直接转换为墙体、地面、门窗和基础家具模型，具有可控性强、运行速度快和离线可用等优点，但视觉真实感和细节丰富度通常有限。

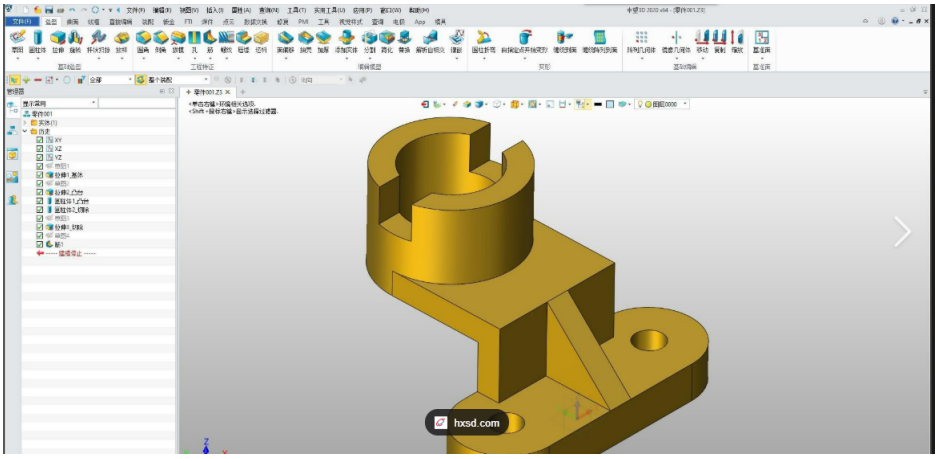


图 2.1　现有三维建模方法：利用建模软件

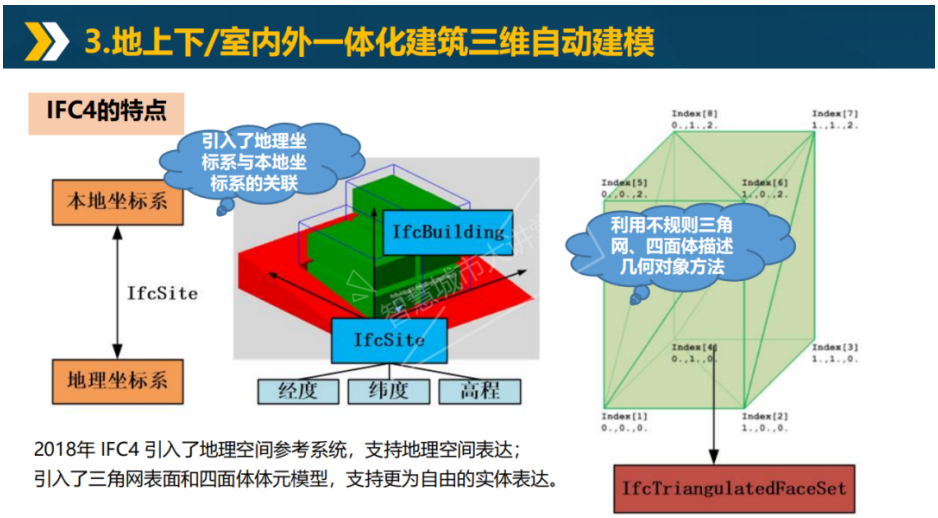


图 2.2　现有三维建模方法：IFC4 建筑信息模型

三维生成大模型为户型三维展示提供了新的技术路径。该类模型通常基于大规模三维数据训练，结合文本、图像或多模态输入，自动生成带有几何形体、纹理材质和视觉细节的三维资产。与传统人工建模相比，三维大模型降低了建模门槛；与纯程序化建模相比，其生成结果通常具有更丰富的材质和外观表现。然而，三维大模型也存在几何可控性不足、生成结果与输入布局不完全一致、依赖云端服务和生成时间不稳定等问题。因此，在户型自动生成系统中引入三维大模型时，需要同时考虑视觉效果、几何准确性和工程稳定性。

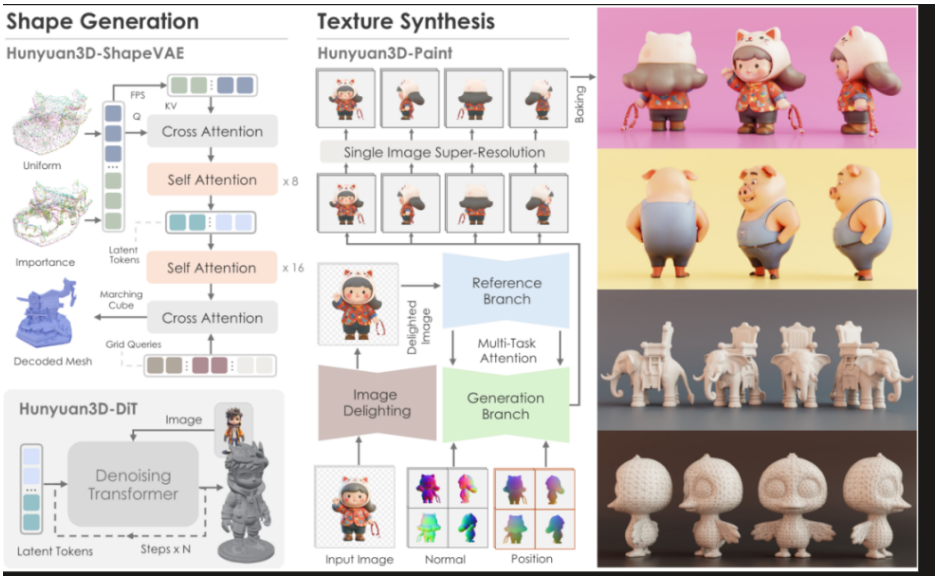


图 2.3　混元 3D 大模型生成原理示意图

本系统采用云端三维生成与本地数据转换相结合的方式。系统对接腾讯混元生 3D 大模型接口，将整数规划求解得到的户型平面布局转换为可用于三维生成的结构化描述。该模块以 Python 网络请求为基础完成云端任务提交、任务状态查询、结果轮询和模型文件下载，并通过统一客户端封装屏蔽底层 API 调用细节，使主界面可以直接调用三维生成能力。

在输入数据处理方面，系统首先读取优化结果中的建筑边界、房间坐标、房间尺寸和房间类型等信息，然后构造适合三维生成模型理解的提示词。提示词不仅包含各房间空间位置和面积信息，还包含房间功能、相邻关系、整体户型轮廓和期望建筑风格等约束条件，以提高生成模型与二维户型结果之间的一致性。系统同时支持文生 3D、图生 3D 和图文混合三种模式，其中图文混合模式将二维户型截图与结构化提示词共同提交给模型，用图像传递空间拓扑关系，用文本补充房间语义与风

格要求。由于三维模型生成通常属于异步任务，系统设计了轮询机制持续查询任务状态，并在任务完成后自动解析返回结果中的模型下载地址，将生成的模型文件保存到本地，便于后续展示、查看和复用。

数据处理：系统使用 Python 内置的 json 模块进行序列化与反序列化，并使用 dataclasses 模块简化数据类定义，从而减少额外依赖并提高代码可维护性。

2.5 本章小结

本章围绕系统实现所需的理论和技术基础进行了说明。首先介绍了整数规划的基本形式和二元变量的建模作用，说明其适合描述户型布局中的位置、尺寸、朝向和逻辑选择问题；随后介绍了 Gurobi 求解器及其在变量创建、约束添加、目标函数设置和求解控制方面的使用方式；接着说明了大 M 法在条件约束表达中的作用；最后概述了系统采用的 Python、Tkinter、NumPy、Matplotlib、Gurobi、三维建模技术和腾讯混元 3D 接口等技术栈。这些内容为后续系统需求分析和核心算法设计提供了理论与工程基础。

3 系统需求分析与总体设计

3.1 系统功能需求分析

通过分析现有户型设计工具的功能不足和实际设计场景中的用户需求，本系统需满足以下核心功能需求：

(1) 约束参数录入与编辑。系统应支持用户便捷录入户型设计需求，包括房间名称、尺寸范围、长宽比约束、邻接关系以及靠外墙需求等信息。输入方式不应局限于单一表格，还应提供图形化拖拽编辑能力，以便用户直观构建房间关系图，并支持模板导入功能，使用户能够基于预设方案进行快速修改。

(2) 户型布局自动优化生成。用户完成约束参数录入和建筑边界绘制后，系统应能够在运行优化命令后自动调用整数规划算法，输出每个房间的精确位置和尺寸。生成结果应满足尺寸、边界、邻接、不重叠等核心约束，并能够作为渲染、评估、预算估算和三维建模等后续模块的统一数据输入。

(3) 户型平面图渲染与导出。优化结果以坐标和尺寸数据形式存储，直接阅读不够直观。因此，渲染器需要将数值结果转换为平面图，绘制房间色块、尺寸标注、门窗示意、家具摆位、图例和指北针，并支持导出 PNG、SVG 和 PDF 格式。

(4) 户型质量评估。系统应能从空间利用率、宽高比合规性、动线效率、自然采光、动静分区和邻接满意度等多个维度对生成的户型方案进行自动评分，帮助用户判断方案的优劣。

(5) 日照采光分析。系统应能根据用户指定的地理位置（纬度）、朝向和日期，模拟太阳轨迹并计算每个房间的日照时数和日照强度分布，以热力图和柱状图的形式呈现分析结果。

(6) 装修预算估算。系统应能根据不同装修档次（经济简装、标准精装、高端豪装）的单价体系，结合各房间的实际面积，自动计算装修的总预算、分项预算和各房间明细预算。

(7) 智能户型推荐。系统应能根据用户输入的家庭人数、面积范围、卧室数量、功能偏好和预算档次等需求参数，从预设的户型模板库中智能匹配并推荐最合适的户型方案。

系统功能需求汇总如表 3.1所示。

表 3.1　系统功能需求分析表

功能模块	输入内容	输出或处理结果
约束编辑	房间尺寸、邻接关系、靠边/靠墙要求	规范化约束文件和合法性校验结果
布局优化	建筑边界、房间约束、优化参数	满足约束的房间坐标和尺寸数据
平面渲染	优化生成的矩形房间列表	带标注的户型平面图及导出文件
户型评估	户型布局、房间类型、边界信息	空间利用率、动线效率、采光等评分
日照分析	地理纬度、日期、朝向、户型布局	日照时长、热力图和文本报告
预算估算	房间面积、装修档次、价格数据	总预算、分项预算和房间明细预算
智能推荐	家庭人数、面积、功能偏好、预算	Top-K 推荐方案及匹配理由
3D 建模	户型平面图、提示词、API Key	三维模型任务结果和本地模型文件

3.2 系统非功能需求

除功能需求外，系统还需满足以下非功能需求：（1）求解效率：对于典型住宅户型（5-15 个房间），优化求解应在 60 秒内完成；（2）可用性：图形界面应直观友好，约束编辑操作应提供清晰的操作指引和实时的输入校验；（3）可扩展性：系统模块化设计，各功能模块松耦合，便于后续功能扩展和算法升级；（4）跨平台兼容性：基于 Python 和 Tkinter 实现，支持 Windows、macOS 和 Linux 等主流操作系统。

3.3 系统总体架构

系统采用分层模块化架构，自底向上分为数据层、核心算法层、业务逻辑层和表示层四个层次。

数据层负责管理持久化数据。模板库保存预设方案的房间参数和标签（templates.json），价格数据库保存装修单价和分项比例（renovation_prices.json），用户约束和建筑边界分别存储在.cons 和.boundary 文件中。

核心算法层是系统的计算核心，包含 Floorplan 类实现的整数规划优化引擎。该

层负责任务调度（Tiling 主流程）、变量创建（CreateVariable）、约束建模（Dimension-Constraint、NonOverlapConstraint、AdjacencyConstraint 等七类约束方法）、目标函数构造（SetObjection）和 Gurobi 求解。

业务逻辑层是核心算法层与表示层之间的桥梁，包含 ConstraintEditor 控制器（管理约束数据在各编辑器组件之间的同步）、FloorplanEvaluator 评估器（协调六大评估指标的运行）、DaylightSimulator 模拟器、RenovationEstimator 估算器、RecommendationEngine 推荐引擎和 Hunyuan3DClient 客户端。系统还提供项目管理功能，用于组织工程文件、历史结果和相关配置。

表示层是用户与系统交互的界面，包含主窗口 MainGUI（Canvas 户型渲染区 + 功能面板）、ConstraintEditorWindow 约束编辑窗口（Notebook 多标签页管理房间、邻接、扩展邻接和靠边约束）、DaylightWindow 日照模拟窗口、RenovationWindow 预算估算窗口、EvaluationWindow 评估报告窗口和 RecommendationWindow 推荐向导对话框。

3.4 系统模块划分

根据功能内聚原则，系统划分为以下七大功能模块：

(1) 约束编辑模块（modules/constraint_editor.py）。提供房间尺寸表格编辑器（RoomTableEditor）、邻接关系图编辑器（AdjacencyEditor）、扩展邻接关系编辑器（ExtendedAdjacencyEditor）和靠边/靠墙房间编辑器（RoomBoundaryEditor），以及约束校验器（ConstraintValidator）和约束文件的导入导出功能。

(2) 核心优化模块（core/floorplan.py）。该模块是整数规划优化算法的实现主体，覆盖变量定义、约束建模、目标函数构造以及调用 Gurobi 求解的完整流程。

(3) 户型渲染模块（modules/renderer.py）。该模块包含用于绘制房间色块的 RoomRenderer、用于尺寸标注的 DimensionAnnotator、用于门窗绘制的 DoorWindowRenderer、用于家具示意的 FurnitureSuggester、用于图例和指北针绘制的 LegendRenderer，以及用于多方案并排比较的 MultiSchemeComparer 等子渲染器。

(4) 日照模拟模块（modules/daylight_simulator.py）。该模块基于天文学模型计算太阳位置，得到各房间的日照时长和光照强度，并输出热力图和柱状图。

(5) 预算估算模块（modules/renovation_estimator.py）。该模块维护不同装修档次的单价表，根据房间类型和面积计算预算，并生成分项预算图。

(6) 智能推荐模块（modules/recommendation_engine.py）。该模块根据家庭人数、面积、卧室数量等需求，从模板库中进行多维度匹配并推荐合适户型。

(7) 3D 建模模块（modules/3d_model.py）。该模块对接腾讯混元 3D 大模型 API，将户型数据转换为结构化三维模型描述，并提交云端生成任务。

此外，系统还包含户型评估模块（modules/floorplan_evaluator.py）、国际化模块(i18n/)、数据读写模块（data/）等辅助模块。

3.5 系统界面布局设计

系统主界面由三部分组成：顶部菜单栏、左侧户型绘制 Canvas 区域（支持显示多边形边界）以及右侧功能按钮区。功能按钮区包含运行优化、显示标注、日照模拟、预算估算、方案对比、导出和 3D 建模等操作入口，下方展示求解历史列表。约束编辑器采用独立弹窗形式，通过 Notebook 多标签页组织房间尺寸、邻接关系、扩展邻接和靠边/靠墙四个面板，左侧提供模板选择和预览区。整体界面采用深蓝暗色系配色（#080c14 背景与 #00e5ff 强调色），以增强系统的科技感和辨识度。相关系统界面截图将在第 5 章系统详细实现部分展示。

3.6 本章小结

本章从系统建设目标出发，对住宅户型自动生成系统进行了需求分析和总体设计。首先明确了约束参数编辑、布局自动生成、平面图渲染、质量评估、日照分析、预算估算和智能推荐等核心功能需求，并提出了求解效率、可用性、可扩展性和跨平台兼容性等非功能需求。随后给出了系统的分层架构设计，将系统划分为数据层、核心算法层、业务逻辑层和表示层，并进一步说明了各功能模块的职责和界面布局。上述设计为后续详细实现提供了清晰的模块边界和开发依据。

4 基于整数规划的户型布局优化算法

本章重点阐述系统的核心算法，即如何将住宅户型布局问题转化为整数规划模型并进行求解。算法的理论基础参考了 Wu 等人[18] 提出的布局优化框架，本文在该框架基础上完成了面向住宅户型生成场景的工程实现，并结合系统功能需求进行了相应扩展。

4.1 问题形式化定义

住宅户型布局优化通常可以采用规则搜索、元启发式优化、约束规划和数学规划等技术路线。规则搜索方法实现较为直接，但依赖人工经验规则，难以在复杂约束条件下保证全局质量；元启发式算法能够在较大解空间中搜索可行布局，但结果稳定性和最优性保证相对有限；约束规划适合描述离散逻辑关系，但在处理连续尺寸、面积利用率和多目标权衡时通常需要额外建模。相比之下，整数规划能够将房间位置、尺寸、朝向、邻接和边界关系统一表示为决策变量与线性约束，并由成熟求解器完成可行性搜索和目标优化，因此适合本文所研究的住宅户型自动生成任务。

Wu 等人提出的布局优化框架为本文算法设计提供了重要参考，其核心思想是将房间抽象为轴对齐矩形，并通过优化模型同时确定房间位置和尺寸。本文在该思路基础上进行了面向工程系统的扩展：一方面保留整数规划模型的可解释性和约束可验证性，另一方面结合住宅户型设计中的玄关、走廊、采光、楼梯和更新编辑等实际需求，引入若干新的约束类型和求解模式，使算法更适合交互式软件系统使用。

给定一个矩形建筑边界 $B = (x_B, y_B, w_B, h_B)$，其中 (x_B, y_B) 为边界左下角网格坐标，w_B 和 h_B 分别为边界的网格宽度和高度。给定 n 个房间的需求规格列表 $R = \{r_1, r_2, \ldots, r_n\}$，每个房间 r_i 包含以下约束参数：房间名称 $label_i$、最小高度 h_i^{min}、最小宽度 w_i^{min}、最小宽高比 ρ_i^{min}、最大宽高比 ρ_i^{max}、目标高度 h_i^{obj} 和目标宽度 w_i^{obj}。

户型布局问题的目标是为每个房间 r_i 确定一个轴对齐矩形的位置 (x_i, y_i) 和尺寸 (w_i, h_i)，使得所有建筑约束得到满足，同时优化整体的空间利用效率。

每个房间矩形由以下四个整数变量描述：x_i（矩形左下角的 x 坐标）、y_i（矩形左下角的 y 坐标）、w_i（宽度）、h_i（高度），以及一个二元变量 $o_i \in \{0, 1\}$（朝向变量，$o_i = 0$ 表示正常朝向即宽度 < 高度，$o_i = 1$ 表示旋转 90 度即宽度 > 高度）。

4.2 变量定义与初始化

系统将建筑边界离散化为网格，每个网格单元的物理尺寸由离散化参数 d（discretization，默认 0.35 米/格）控制。所有坐标和尺寸均以网格单元为单位。对于第 i 个房间，定义如下 Gurobi 决策变量：

- `y_i`（对应 var0）：房间左下角 y 坐标，整数变量，范围 $[y_B, y_B + h_B]$
- `x_i`（对应 var1）：房间左下角 x 坐标，整数变量，范围 $[x_B, x_B + w_B]$
- `h_i`（对应 var2）：房间高度，整数变量，下限为房间类型对应的最小尺寸
- `w_i`（对应 var3）：房间宽度，整数变量，下限为房间类型对应的最小尺寸
- `ori_i`（对应 orientation）：房间朝向，二元变量（0 或 1）

最小尺寸按房间类型区分：走廊（标签含"CORRIDOR"）至少 $0.8/d$ 格，楼梯（标签含"STAIRS"）至少 $0.7/d$ 格，普通房间至少 $1.5/d$ 格。采用整数变量后，房间边界能够与离散网格对齐，从而简化邻接和不重叠关系的判定。

4.3 约束建模

本系统共包含七大类建筑约束，以下逐一详述其数学表达和 Gurobi 实现方式。

4.3.1 尺寸约束与朝向控制

尺寸约束是保证每个房间满足基本使用功能的基础约束，包含三个子约束：

最小尺寸约束：利用朝向二元变量 o_i，实现了旋转感知的最小尺寸控制：

$$\begin{aligned} h_i &\geq h_i^{min} + (w_i^{min} - h_i^{min}) \cdot o_i \\ w_i &\geq h_i^{min} + (w_i^{min} - h_i^{min}) \cdot (1 - o_i) \end{aligned} \tag{4.1}$$

当 $o_i = 0$（正常朝向）时，高度约束取 $h_i \geq h_i^{min}$，宽度约束取 $w_i \geq w_i^{min}$；当 $o_i = 1$（旋转 90 度）时，两者的取值互换，实现了旋转感知。

宽高比约束：利用大 M 法控制房间的宽高比在指定范围内：

$$\begin{aligned} \rho_i^{min} \cdot h_i &\leq w_i + M \cdot o_i \\ \rho_i^{max} \cdot h_i &\geq w_i - M \cdot o_i \\ \rho_i^{min} \cdot w_i &\leq h_i + M \cdot (1 - o_i) \\ \rho_i^{max} \cdot w_i &\geq h_i - M \cdot (1 - o_i) \end{aligned} \tag{4.2}$$

边界内部约束：确保所有房间完全位于建筑边界之内：

$$y_i + h_i \leq y_B + h_B, \quad x_i + w_i \leq x_B + w_B \tag{4.3}$$

4.3.2 不重叠约束

对于任意两个不同的房间 i 和 j，两者必须在至少一个方向上不重叠。使用大 M 法和四个辅助二元变量实现：

$$\begin{aligned} y_j + h_j &\leq y_i + M \cdot (1 - \delta_0) \\ y_j &\geq y_i + h_i - M \cdot (1 - \delta_1) \\ x_j + w_j &\leq x_i + M \cdot (1 - \delta_2) \\ x_j &\geq x_i + w_i - M \cdot (1 - \delta_3) \\ \delta_0 + \delta_1 + \delta_2 + \delta_3 &\geq 1 \end{aligned} \tag{4.4}$$

其中 $\delta_k \in \{0, 1\}$ 是专为房间对 (i, j) 创建的四个二元变量，各自对应一个方向关系：j 在 i 的下方、上方、左侧、右侧。最后那条 $\sum \delta_k \geq 1$ 保证至少有一个方向的分离关系成立。

类似的不重叠约束同样适用于房间与障碍物（如楼梯间占位）之间，确保障碍物区域不被任何房间占据。

4.3.3 邻接约束

邻接约束确保指定的两个房间共享一段公共边界。对于普通邻接关系，要求两个房间在水平或垂直方向上有至少 $1.35/d$ 格长度的边界接触：

$$\begin{aligned} y_i + h_i &\geq y_j + L_{min} \cdot \tau \\ y_i &\leq y_j + h_j - L_{min} \cdot \tau \\ x_i + w_i &\geq x_j + L_{min} \cdot (1 - \tau) \\ x_i &\leq x_j + w_j - L_{min} \cdot (1 - \tau) \end{aligned} \tag{4.5}$$

其中，$\tau \in \{0, 1\}$ 控制接触方向（$\tau = 1$ 为垂直方向，$\tau = 0$ 为水平方向），$L_{min} = Round(1.35/d)$ 为最小接触长度。

玄关 (FOYER) 和走廊 (CORRIDOR) 之间的 T 型邻接是个特例——不是简单的边对边接触，而是要求玄关的一端顶在走廊某条边的中间位置。系统用 TobeTShape

方法处理，两个接触变量 $touch_1$ 和 $touch_2$ 满足 $touch_1 + touch_2 = 1$，保证衔接处形成标准的 T 型。

4.3.4 边界约束

边界约束要求指定房间（通常为需要采光的房间，如客厅、主卧、次卧等）紧贴建筑外边界。通过四个二元变量 t_0, t_1, t_2, t_3 控制房间贴靠边界的方向，配合障碍物接触处理：

$$\begin{aligned} y_i &\le y_B + M \cdot (1 - t_0) \\ y_i + h_i &\ge y_B + h_B - M \cdot (1 - t_1) \\ x_i &\le x_B + M \cdot (1 - t_2) \\ x_i + w_i &\ge x_B + w_B - M \cdot (1 - t_3) \\ t_0 + t_1 + t_2 + t_3 + \sum_k t_{obs}^k &= 1 \end{aligned} \tag{4.6}$$

其中，对于有障碍物的一面，还需通过额外的二元变量 t_{obs}^k 处理与障碍物外侧的接触。

4.3.5 扩展邻接约束

扩展邻接约束要解决的场景是：一个房间（通常是走廊）需要同时跟两个房间邻接——比如卧室 A 和卧室 B 分布在走廊两侧。系统引入三个二元变量 $touch_0$、$touch_1$ 和 $touch_2$，$touch_2$ 用来决定走廊先跟哪个房间对上。

4.3.6 靠墙约束

靠墙约束不同于边界约束，它要求房间贴靠特定的内墙而非外墙。通过遍历预定义的墙体线段列表，为每个墙体创建一个二元接触变量，要求至少有一个 $\exp = \sum touch_{wall} \ge 1$。

4.3.7 其他约束

走廊最大尺寸约束：走廊宽度或高度不超过 $1.5/d$ 格，避免走廊过宽浪费面积：

$$h_{corr} \le 1.5/d + (M - 1.5/d) \cdot o_{corr} \tag{4.7}$$

二层楼梯约束：对于两层住宅的生成，一层楼梯间必须位于二层投影区域（secondOnFirst）之内，且不与区域内障碍物重叠。二层走廊必须与楼梯间的出口侧邻接，系统针对楼梯间的四种位姿（宽型左出口、宽型右出口、高型上出口、高型下出口）分别设计了专门的约束逻辑。

4.4 目标函数构造

本系统的目标函数由三个加权惩罚项组成：

面积惩罚项（Area Penalty）：

$$E_{area} = w_{area} \cdot \left(w_B \cdot h_B - \sum_k A_{obs}(k) - \sum_{i=1}^{n} h_i \cdot w_i \right) \tag{4.8}$$

其中，w_{area} 为面积权重（默认 1.0），$A_{obs}(k)$ 为障碍物面积。该惩罚项推动所有房间尽可能填满建筑边界内的可用空间。

尺寸偏差惩罚项（Error Penalty）：

$$\begin{aligned} E_{error} = w_{error} \cdot \sum_{i=1}^{n} \Big[& (h_i - o_i \cdot w_i^{obj} - (1-o_i) \cdot h_i^{obj})^2 \\ & + (w_i - (1-o_i) \cdot w_i^{obj} - o_i \cdot h_i^{obj})^2 \Big] \end{aligned} \tag{4.9}$$

该二次惩罚项度量了各房间实际尺寸与目标尺寸之间的差异，w_{error} 为误差权重（默认 1.0）。二次形式的目的是使目标函数保持凸性，同时惩罚与目标尺寸偏离较大的方案。

辅助面积惩罚项（Extra Area Penalty）：

$$E_{extra} = w_{extra} \cdot \sum_{i \in \{corridor, stairs\}} h_i \cdot w_i \tag{4.10}$$

额外惩罚走廊和楼梯的面积（因为这些功能空间的面积应尽量控制），w_{extra} 为辅助权重（默认 0.0，通常不启用）。

总目标函数为三个惩罚项的加权和：

$$\min \ E = E_{area} + E_{error} + E_{extra} \tag{4.11}$$

4.5 求解流程

算法的主入口为 Floorplan.Tiling() 方法，其执行流程如算法 1所示。

算法 1 整数规划户型优化求解主流程

输入： 建筑边界 B、房间约束列表 R

输出： 各房间矩形位置和尺寸列表 $\{R_i(x_i, y_i, w_i, h_i)\}$

初始化 Gurobi 模型参数（时间限制 500 秒，MIPFocus=2，控制台日志关闭）
遍历 R 中每个房间，创建 5 个决策变量（x, y, w, h, orientation）
添加尺寸约束：最小尺寸 + 宽高比 + 边界内部
添加不重叠约束：房间-房间 + 房间-障碍物
if 生成模式（非更新模式） **then**
    添加邻接约束和扩展邻接约束
    添加靠边约束和靠墙约束
    添加其他约束（走廊/楼梯尺寸限制）
    构造并设置二次目标函数
    输出模型文件（Generate.lp）
    调用 model.optimize() 求解
else
    构造并设置目标函数（仅面积项）
    调用 model.optimize() 求解
end if
从求解结果中提取每个变量的最优值
构建 RECTROOM 对象列表并返回

求解器配置方面，系统设置了以下关键参数以优化求解性能：Heuristics=0.1（适度使用启发式策略以加速找到初始可行解）、MIPFocus=2（优先寻找高质量的可行解而非急于证明最优性）、Presolve=2（启用激进预处理以削减问题规模）、Cuts=0（禁用割平面生成以减少每次迭代的求解开销）、TimeLimit=500（设置 500 秒的时间上限防止求解时间过长）。

4.6 更新模式

除了从头生成户型（GenerateGeneral 模式）外，系统还支持更新模式（UpdatePreserve 和 UpdateMerge），用于对已有布局方案进行局部调整。在更新模式下，系统将现有的每个房间沿某一方向（水平或垂直）分割为两个子房间，然后按照分割方向施加保持约束（Preserve）或合并约束（Merge），允许子房间在限定范围内微调位置和尺寸，最终仍通过整数规划求解给出更新后的布局。该功能在需要保留原有布局主体结构同时进行局部优化时非常有用。

4.7 算法扩展与改进

相较于参考布局优化框架，本文实现的算法并非仅进行简单复现，而是在约束类型、房间类型处理、门位置约束和更新模式等方面进行了工程化扩展。

首先，系统新增了 T 形邻接约束。普通邻接约束主要描述两个房间沿一条边平行贴靠的情形，而住宅户型中玄关与走廊之间常出现 T 形连接，即一个房间的一端接触另一个房间的中部区域。为描述该关系，系统在 `TobeTShape()` 中引入两个二元接触变量 $touch_1$ 和 $touch_2$，并添加 $touch_1 + touch_2 = 1$ 约束，保证恰好选择一种 T 形接触方向，从而提高走廊型空间组织的表达能力。

其次，系统增加了 Room-Wall 靠墙约束。参考模型更多关注房间之间以及房间与外边界之间的关系，而实际设计中还可能要求某些房间贴靠预定义内墙或特定墙体线段。为此，系统使用 `DIRECTEDLINE` 描述墙体线段，并在 `TobeRoomWall()` 中为每条候选墙体创建接触变量，通过 $\sum touch_{wall} \geq 1$ 保证指定房间至少贴靠一面预定义墙。该约束增强了系统对局部设计意图和既有墙体条件的适应能力。

第三，系统扩展了入户门位置约束。原有位置约束主要面向点位置控制，而本系统将其扩展为有向线段约束，用于精确描述门的位置、方向和朝向。`TobePosition()` 根据门线段的水平/垂直方向以及门位于房间上、下、左、右侧的情况，分别构造约束条件，使玄关等入口空间能够与真实入户门位置保持一致。

第四，系统对不同房间类型采用特化尺寸约束。普通房间、走廊和楼梯在功能上具有不同尺度要求，若统一使用相同最小尺寸，容易导致走廊过宽或楼梯空间不合理。因此系统设置普通房间最小尺寸、走廊最小/最大尺寸和楼梯最小/最大尺寸，并在变量创建和其他约束中分别处理。例如走廊采用 $0.8m$ 最小尺寸和 $1.5m$ 最大尺寸控制，楼梯采用 $0.7m$ 最小尺寸和 $1.2m$ 最大尺寸控制，从而使生成结果更符合住宅空间尺度要求。

最后，系统新增了面向已有布局编辑的更新模式。传统从零生成模式适合首次生成户型，但在实际交互过程中，用户往往希望保留已有布局主体，仅对局部区域进行调整。系统因此设计了 UpdatePreserve 和 UpdateMerge 两种模式：前者在较小扰动范围内保持原有位置和尺寸关系，适合微调；后者允许在更大范围内进行合并和调整，适合局部重构。通过将已有房间沿水平或垂直方向分割为子房间并施加保持或合并约束，系统能够在延续原方案结构的同时完成局部优化。

4.8 三维建模数据转换与生成策略

整数规划模型输出的是二维网格上的矩形房间集合，其结果本质上仍是平面布局数据。为了使用户能够更直观地理解空间尺度、房间关系和整体居住体验，系统在二维布局求解之后增加了三维建模模块，将优化结果转换为可提交给云端三维生成模型的结构化描述，并支持文生 3D、图生 3D 和文图混合 3D 三种生成方式。



图 4.1　二维户型结果到三维建模输入的转换流程

该模块的核心问题不是简单调用外部接口，而是如何将整数规划结果中的精确几何信息转译为生成模型能够理解的输入。对第 i 个房间，求解器输出矩形 $R_i = (x_i, y_i, w_i, h_i)$，其中 (x_i, y_i) 为屏幕坐标系中的左上角位置，w_i 和 h_i 分别为网格宽度和高度。由于二维绘图界面通常采用左上角为原点、y 轴向下的屏幕坐标，而三维建模提示词更适合使用左下角为原点、y 轴向上的数学坐标，因此系统首先进行坐标系转换：

$$x_i' = x_i, \quad y_i' = H_{grid} - y_i \tag{4.12}$$

其中 H_{grid} 为当前户型网格的总行数。对房间矩形的四个角点，系统分别计算左

上、右上、右下和左下坐标，并将其写入提示词。以房间 R_i 为例，其屏幕坐标角点为 (x_i, y_i)、$(x_i + w_i, y_i)$、$(x_i + w_i, y_i + h_i)$ 和 $(x_i, y_i + h_i)$，经过上述转换后得到数学坐标系下的四角点描述。该处理保证了三维生成模型接收到的不只是房间名称和大致面积，而是包含明确位置、尺寸和拓扑关系的二维几何约束。

除房间明细外，系统还会将建筑边界多边形写入提示词。当边界对象中存在 `boundingPoint` 时，模块遍历所有边界顶点并进行同样的坐标变换，形成“顶点序列”描述；同时给出离散化比例，例如“每格等于 0.35 米”。最终提示词由三部分组成：第一部分说明生成目标和风格要求，如现代住宅、白色墙体、木地板和典型家具；第二部分说明统一坐标系和比例尺；第三部分列出建筑边界与每个房间的四角点坐标。其生成流程可概括如下：

算法 2 二维户型结果到三维建模提示词的转换流程

输入： 房间矩形列表 $\{R_i\}$、建筑边界 B、离散化比例 d、网格总行数 H_{grid}
输出： 三维建模提示词 P

初始化提示词 P，写入模型风格、材质和家具要求
写入坐标系约定：左下角为原点，x 轴向右，y 轴向上
if 建筑边界顶点存在 **then**
  遍历边界顶点 (x, y)，计算 $(x, H_{grid} - y)$
  将转换后的顶点序列写入 P
end if
for 每个房间 $R_i = (x_i, y_i, w_i, h_i)$ **do**
  计算房间四个角点并转换到数学坐标系
  写入房间名称、四角点坐标和网格尺寸
end for
写入比例尺 d 和墙体、门、家具等语义补充
return P

在云端接口层，系统封装了 `Hunyuan3DClient` 客户端类。该类负责 API Key 认证、任务提交、状态查询、结果轮询和模型文件下载。腾讯混元生 3D 接口与常见 OpenAI 兼容接口存在差异：认证头中 `Authorization` 字段直接填写 `sk-xxx` 格式的密钥，不附加 `Bearer` 前缀；提交任务使用`/v1/ai3d/submit` 端点，查询任务使用`/v1/ai3d/query` 端点。由于接口返回可能存在嵌套结构和扁平结构两种形式，系统通过 `_normalize_response()` 方法将返回结果统一整理为包含 `JobId`、`Status` 和 `Progress` 等字段的字典，降低后续状态处理复杂度。

任务执行采用异步轮询机制。系统提交任务后首先获得 JobId，随后每隔 5 秒查询一次任务状态，最长等待时间设置为 30 分钟。查询过程中，GUI 层通过进度回调刷新进度条和状态文本；若任务成功完成，系统递归扫描返回结果中的 Url、DownloadUrl、GlbUrl、ModelUrl 等字段，提取可下载的模型链接并保存为本地文件。该设计将长耗时的云端生成过程与本地交互界面解耦，避免界面阻塞。

在生成模式设计上，系统最初仅采用文生 3D 方式，即把上述坐标化提示词直接提交给三维生成模型。然而在开发测试中发现，纯文本并不是传递空间布局信息的稳定载体。对于同一段包含房间坐标和尺寸的文字描述，不同大模型对空间关系的理解能力存在明显差异：部分模型能够较好还原房间位置、大小和邻接关系，而另一些模型则容易出现房间错位、数量不一致或拓扑关系错误的问题。该现象说明，二维户型布局虽然可以线性化为文字，但文本序列对空间关系的表达效率较低，模型需要从坐标和自然语言中重新推理平面拓扑，结果具有不确定性。

基于这一工程观察，系统在文生 3D 之外增加了图生 3D 和文图混合 3D 模式。图生 3D 模式直接使用当前二维户型截图作为输入，使模型能够从图像中获取空间拓扑；文图混合模式则同时提交二维户型截图和结构化提示词，其中图像提供房间边界、相对位置和整体轮廓，文字补充房间名称、材质风格、家具布置和比例尺等语义信息。与纯文本模式相比，文图混合模式把空间理解的主要信息通道从线性文本转移到二维图像，同时保留文字对功能语义的表达能力，因此更适合本系统中“精确二维布局到三维展示”的任务场景。

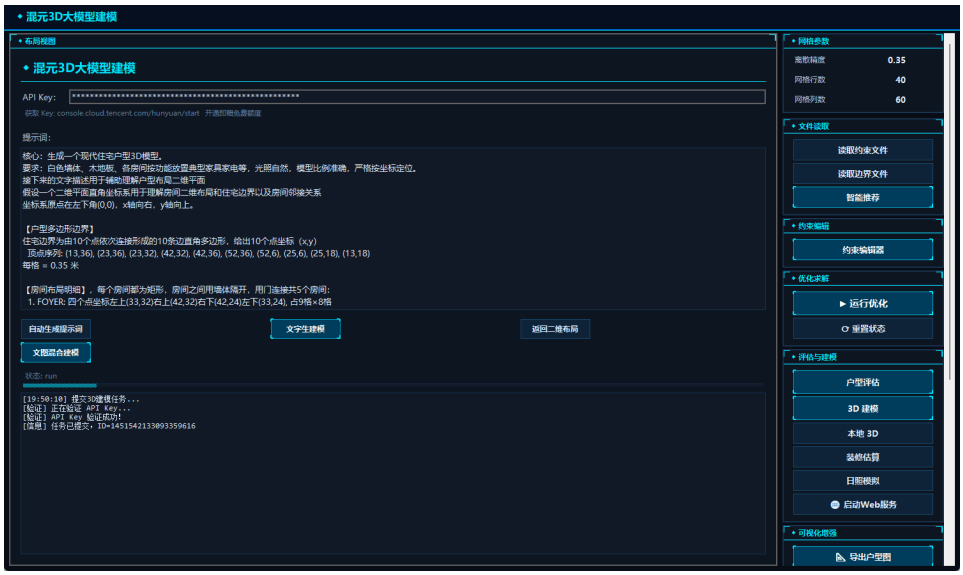


图 4.2　文图混合三维生成模式示意图

需要指出的是，当前三维建模模块仍依赖云端生成模型，其结果质量会受到模型空间理解能力、接口稳定性和任务排队时间等因素影响。文图混合模式能够缓解纯文本空间推理不稳定的问题，但并不能完全保证生成模型严格满足每一项几何约束。后续可进一步探索面向建筑空间的结构化输入格式，例如将房间边界、邻接关系、门窗位置和家具约束组织为 JSON Schema，再与图像输入共同约束三维生成过程。

4.9 本章小结

本章详细阐述了基于整数规划的户型布局优化算法。首先对住宅户型布局问题进行了形式化定义，明确了建筑边界、房间规格、位置尺寸变量和朝向变量等核心要素；随后介绍了变量初始化方式和网格离散化处理方法；在约束建模部分，重点说明了尺寸约束、不重叠约束、邻接约束、边界约束、扩展邻接约束、靠墙约束以及其他特殊约束的数学表达；之后构造了由面积惩罚、尺寸偏差惩罚和辅助面积惩罚组成的目标函数，并给出了完整求解流程和更新模式。最后，本章说明了二维整数规划结果向三维建模输入的转换方法，讨论了坐标系变换、提示词构建、云端任务轮询以及文图混合生成策略。通过本章建模，户型生成问题被转化为可由 Gurobi 求解器处理的优化问题，并进一步形成了面向三维展示的数据基础。

5 系统详细设计与实现

系统主界面采用顶部菜单栏、左侧绘图区域和右侧功能面板的布局方式，系统初始化界面如图 5.1所示。项目管理界面用于组织工程文件、历史结果和配置数据，如图 5.2所示。

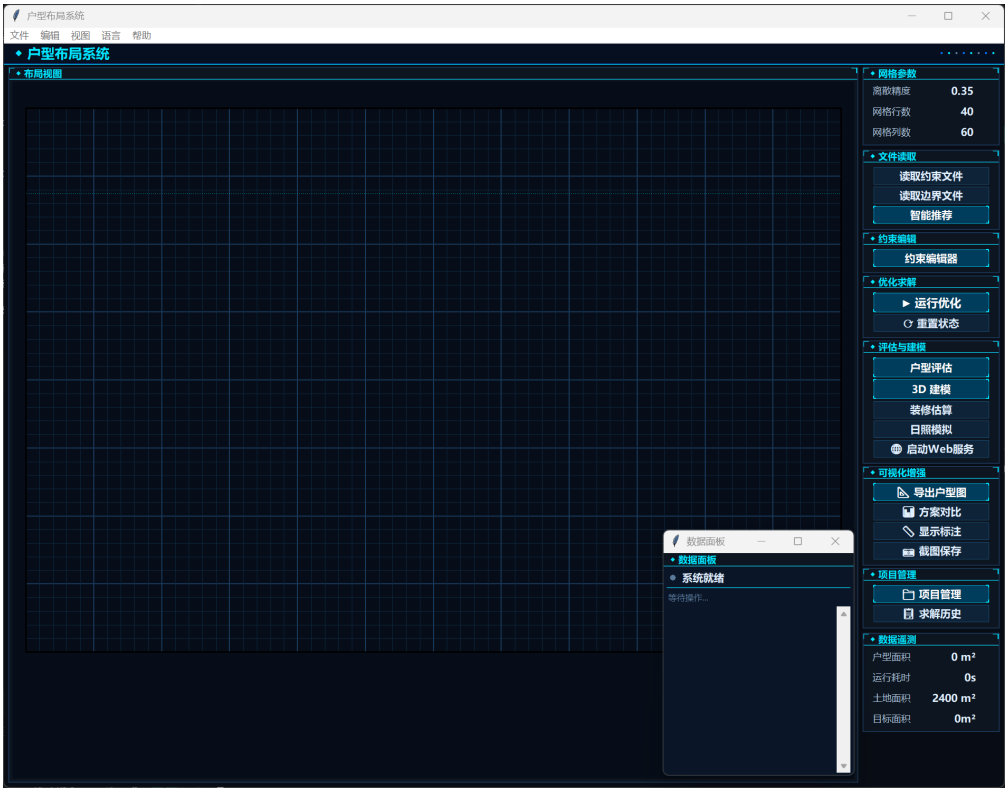


图 5.1　系统主界面

5.1 约束编辑器设计与实现

约束编辑器是用户输入户型设计参数的入口模块，设计为独立的弹出窗口（ConstraintEditorWindow），内部使用 ttk.Notebook 多标签页组织四个编辑子面板。约束文件的二级编辑界面如图 5.4所示。

房间尺寸表格编辑器（RoomTableEditor）基于 ttk.Treeview 实现，支持 18 种预设房间类型（包括玄关、客厅、餐厅、厨房、主卧、次卧、儿童房、书房、主卫、公卫、阳台、洗衣房、储物间、衣帽间、走廊、楼梯、多功能房和杂物间）。表格列包含序号、房间名称（下拉选择）、最小宽高比、最大宽高比、目标高度和目标宽度。双击单元格可进入编辑模式，房间名称列使用 Combobox 提供类型选择并自动填充该



图 5.2　项目管理界面



图 5.3　约束编辑器窗口

类型的推荐参数。底部状态栏实时显示房间总数、目标总面积和尺寸范围。支持添加、删除、上移、下移和右键菜单操作。

邻接关系图编辑器（AdjacencyEditor）提供基于 Canvas 的画布式交互。每个房间表示为一个带颜色的圆角矩形节点，节点可在画布上自由拖拽移动。点击一个节

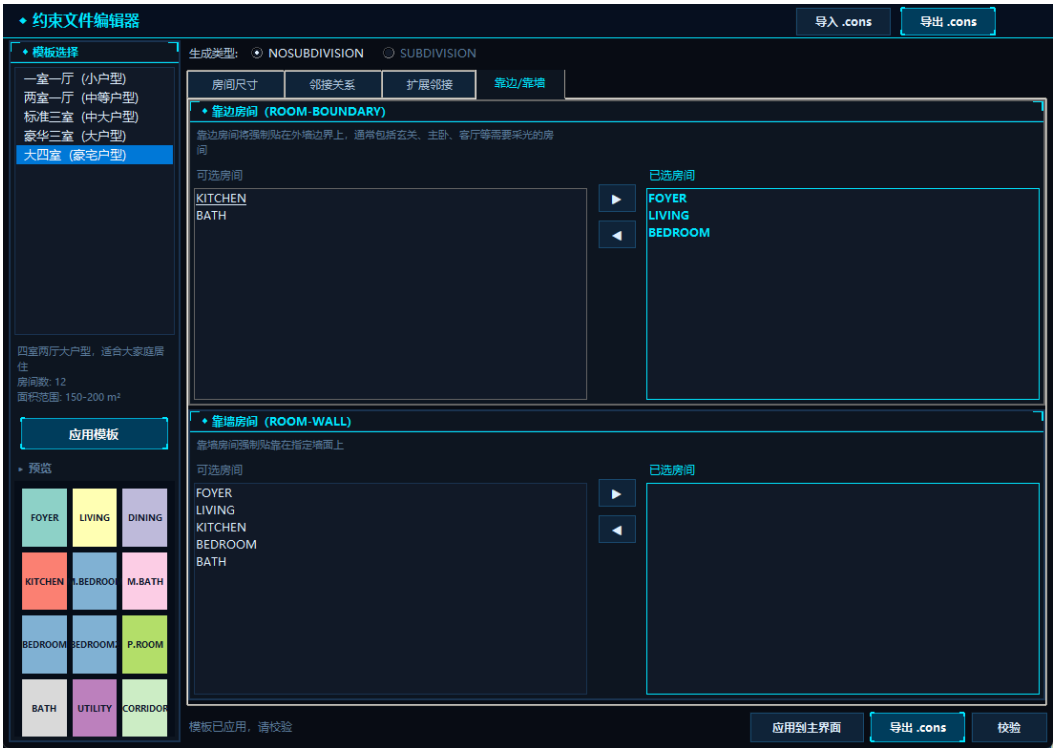


图 5.4　约束文件编辑器二级界面

点后再点击另一个节点即可创建两个房间之间的邻接连线。系统内置简易力导向布局算法（50 次迭代），在节点位置初始化时自动运行，使得有邻接关系的节点相互吸引、无关系的节点相互排斥，从而产生美观的拓扑布局。右侧邻接列表显示当前所有邻接关系，双击可删除。T 型邻接（FOYER-CORRIDOR）以特殊虚线标识。

扩展邻接编辑器（ExtendedAdjacencyEditor）由三个 Combobox 和一个 Listbox 组成。用户分别选择中间房间（如走廊）及两个邻接房间后，即可添加一组三元扩展邻接关系。

靠边/靠墙编辑器（RoomBoundaryEditor）采用双列表对照模式，左侧列表展示可选房间，右侧列表展示已选房间，用户通过左右箭头按钮完成房间选择。靠边房间用于约束需要采光的房间紧贴外墙，靠墙房间则用于约束房间贴靠指定内墙。

约束校验器（ConstraintValidator）在导出或应用约束前自动运行六项检查：房间名称非空且不重复、宽高比最小值不小于 1、最大宽高比不小于最小宽高比、目标尺寸均大于 0、邻接关系中的房间均在房间列表中、扩展邻接关系中的房间均在房间列表中。同时给出信息性提示（如建议添加玄关和走廊）。

5.2 核心优化引擎实现

核心优化引擎的实现类为 Floorplan，位于 core/floorplan.py 文件中，总计约 1060 行代码。该文件完整复刻了 C++ 参考实现中的数据结构、约束建模和求解逻辑。

系统读取建筑边界文件后，会在画布中显示边界轮廓，界面效果如图 5.5所示；在完成约束和边界读取后，调用优化引擎生成户型布局结果，如图 5.6所示。

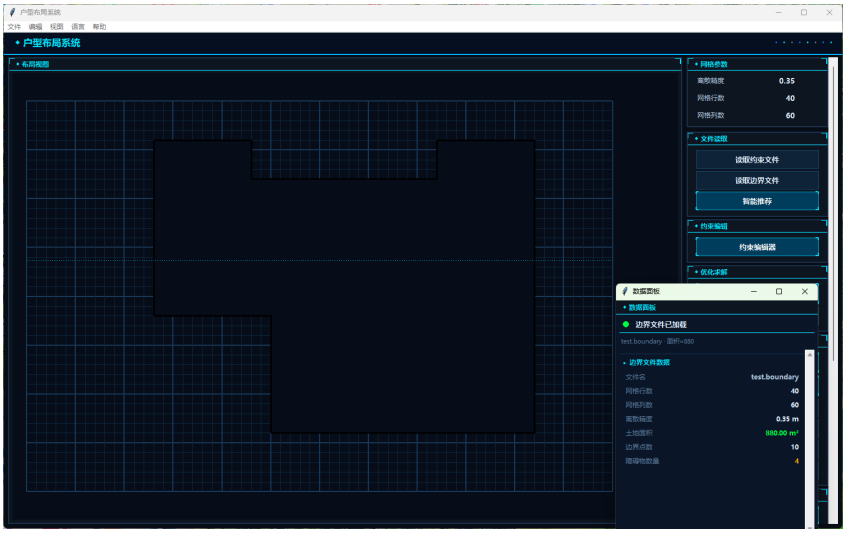


图 5.5　读取边界文件后的界面效果

数据结构体系：系统定义了 11 个关键 dataclass 数据类，包括 FLOATDIMENSION（浮点尺寸规格）、INTDIMENSION（整数离散化后尺寸规格）、DIRECTEDLINE（有向线段，用于门洞和墙体描述）、RECTROOM（矩形房间）、ADJACENCY（二元邻接）、EXADJACENCY（三元扩展邻接）、ROOMWALL（墙体集合）、CONSTRAINT（完整的约束集）、BOUNDARY（建筑边界含障碍物和轮廓点）、ROOMTYPE（含入口信息的房间类型）和 VARIABLE（每房间的 5 个 Gurobi 变量集合）。

全局环境管理：系统使用全局 Gurobi 环境对象 _gp_env（gp.Env()），并由所有 Floorplan 实例共享该对象，从而避免每次创建新实例时重复初始化求解环境。

回调监控：MyCallback 类用于监控求解过程。每当求解器发现新的可行解并触发 MIPSOL 回调时，系统记录当前目标值、最优界和耗时信息，界面效果见图 5.7。此外，系统还设计了基于时间阈值的提前终止机制：在找到第一个可行解之前，最多允许 initial_timeout 秒；找到可行解之后，如果在 mip_timeout 秒内未发现更优解，则提前结束求解。

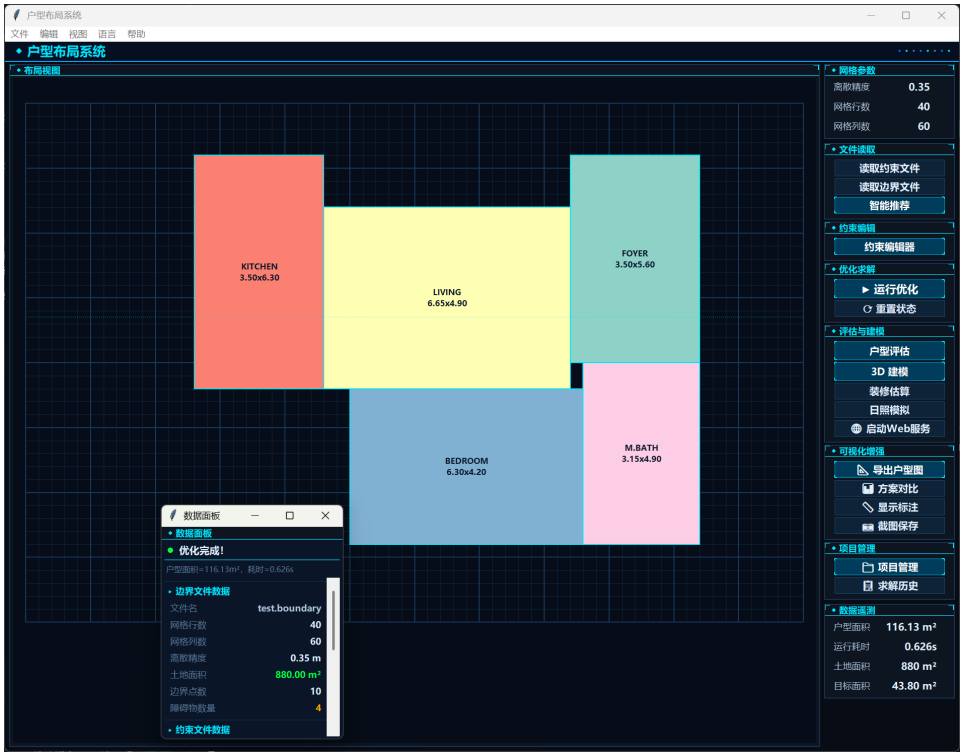


图 5.6　整数规划优化生成的户型布局



图 5.7　求解过程监控历史

离散化与尺度转换：从用户输入的浮点尺寸参数到整数网格坐标的转换由 Round 函数完成（四舍五入到最近整数），并通过面积比例参数 area_ratio（默认 1.0）统一调谐所有尺寸约束。

5.3 户型渲染器实现

渲染器模块（modules/renderer.py，约 1450 行）包含八个子渲染器类和导出器，通过 FloorplanRenderer 主类统一调度。

优化结果经渲染器处理后可生成带尺寸标注的户型平面图，如图 5.8所示。

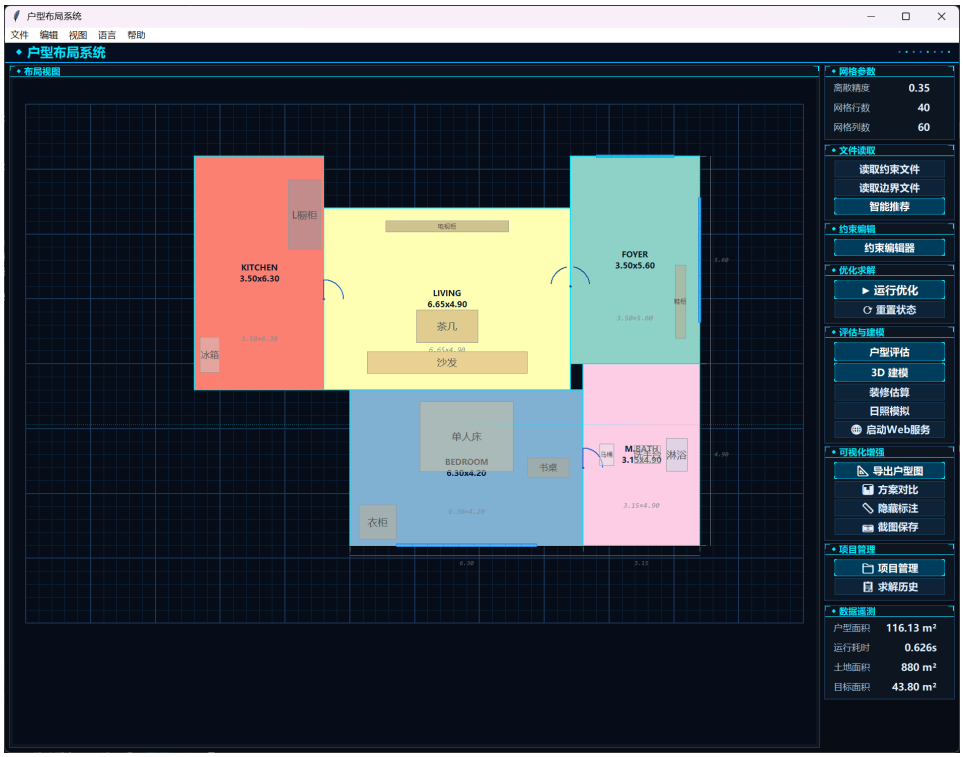


图 5.8　带标注的户型平面图

房间渲染器（RoomRenderer）：使用 Matplotlib 的 FancyBboxPatch 绘制带圆角的矩形房间色块，颜色映射基于 18 种房间类型的预定义配色方案。每个色块内部标注房间中文名称和实际物理尺寸（米）。

尺寸标注器（DimensionAnnotator）：自动识别贴靠外墙的房间，在其外侧绘制尺寸引出线和长度标注（单位：米）。同时标注建筑轮廓的总宽度和总高度。

门窗渲染器（DoorWindowRenderer）：根据邻接关系判断房间之间的共享边界位置，绘制不同类型的门符号（单开门用 90 度弧线 + 指示线、双开门用 180 度对称双弧线、推拉门用双向箭头线）。为靠外墙的房间在外边界上绘制窗户标记（双线 + 横格线）。

家具示意器（FurnitureSuggester）：根据房间类型放置典型家具图标（如客厅的沙发、茶几、电视柜，餐厅的餐桌，厨房的 L 型橱柜和冰箱，卧室的床和衣柜，卫

生间的马桶、洗手台和淋浴），以半透明色块标注。

图例渲染器（LegendRenderer）：在渲染图上方绘制图例（颜色-房间名称对照）、指北针（圆形 + 箭头标注 N/S）和比例尺。

多方案对比器（MultiSchemeComparer）：支持同时渲染多个布局方案，并排展示平面图和六维评分雷达图，便于直观对比。

导出功能（LayoutExporter）：支持 PNG（高分辨率 300dpi 位图）、SVG（矢量图形）和 PDF（可打印文档）三种导出格式，以及通过 ezdxf 库导出 DXF 格式（方便与 AutoCAD 等专业软件交互）。

5.4 日照采光模拟器实现

日照模拟器（modules/daylight_simulator.py，约 700 行）基于天文学太阳位置模型进行计算。

太阳位置计算：根据日期（计算年积日 day_of_year）、纬度（latitude）和小时（hour，范围 6:00-18:00），依次计算太阳赤纬（solar_declination，使用 Cooper 公式[23]）、时角（hour_angle，每小时间隔 15 度）、太阳高度角（sun_altitude，判断是否大于 0 即日出）和太阳方位角（sun_azimuth），得到每个整点时刻的太阳位置。

窗户检测与采光计算：系统自动识别每个靠外墙房间的外侧边界，并在该边界上设置外窗。随后利用窗户朝向与太阳方向向量的点积判断是否存在直射阳光（阈值设为 0.3），并结合太阳高度角的正弦值进行光照衰减计算。

可视化输出：系统使用 imshow 绘制日照热力图，并叠加房间边界和建筑轮廓线，同时在侧边展示各房间日照时数柱状图，并支持导出文本报告。

用户界面：DaylightWindow 提供朝向选择（8 个方向）、日期选择（夏至/春分/秋分/冬至预设）、纬度输入和运行模拟按钮。模拟结果以左右分栏形式展示热力图和柱状图，如图 5.9所示。

5.5 装修预算估算器实现

预算估算器（modules/renovation_estimator.py，约 630 行）实现如下功能：

三级装修档次体系：经济简装（1000 元/平方米）、标准精装（2000 元/平方米）、高端豪装（4000 元/平方米），每种档次定义了 18 种房间类型的单价调整系数和 9 项工程类别的费用占比。

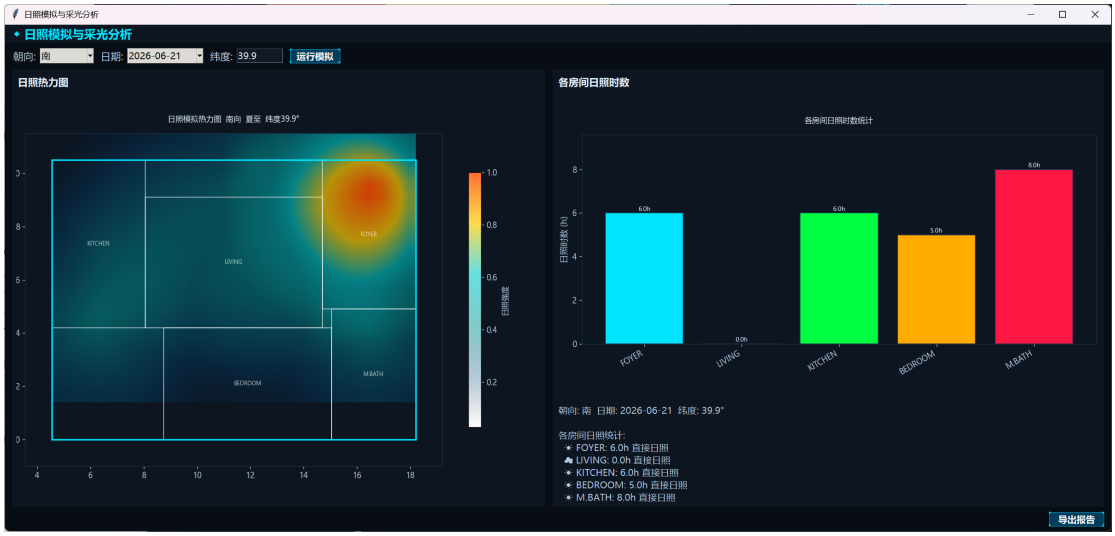


图 5.9　日照采光模拟界面

预算计算逻辑：对每个房间，系统根据其标签（如 KITCHEN）匹配对应房间类型，并读取单价调整系数，按照“面积 × 基准单价 × 调整系数”计算房间预算。在得到总预算后，系统根据预定义比例拆分为基础工程、水电、墙面、地面、厨房、卫浴、门窗、灯具五金和软装预留等 9 个分项。

细项展开：系统对厨房和卫生间进行更细粒度的预算拆分，包括橱柜、烟机灶具、马桶和花洒等项目，并为每一项配置单价和估算用量。

可视化输出：系统左侧展示分项预算水平柱状图，右侧展示费用占比饼图，下方展示各房间预算柱状图，见图 5.10。参考图中的预算估算界面如图 5.11所示，支持展示不同装修档次下的总价、分项预算和房间明细，并支持导出文本预算报告。



图 5.10　装修预算估算界面

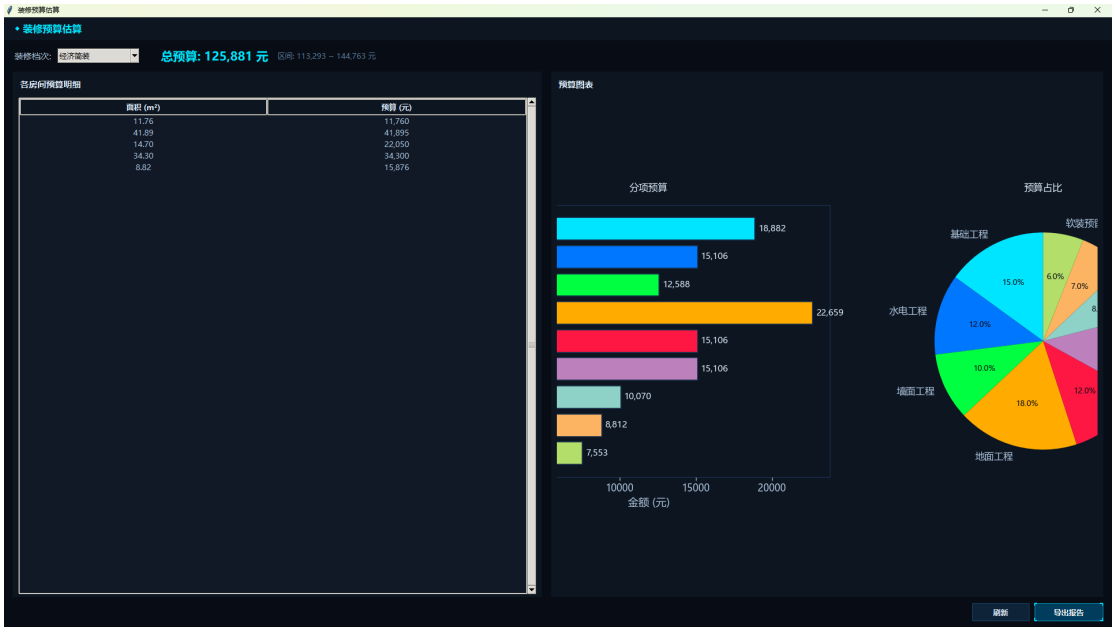


图 5.11　装修预算估算参考界面

5.6 智能推荐引擎实现

推荐引擎（modules/recommendation_engine.py，约 650 行）采用“候选方案生成—多维评分—Top-K 排序”的算法流程。该模块并非简单从固定模板库中读取若干预设方案，而是根据用户需求动态生成候选户型配置，并通过评分函数对候选方案进行排序推荐。

用户需求建模：UserRequirement 数据类包含家庭人数、面积范围、卧室数量、是否需要书房、是否需要餐厅、预算档次、朝向偏好和附加房间列表等维度。其中，家庭人数用于推断最少卧室数量，面积范围用于面积匹配评分，卧室数量用于方案合理性判断，预算档次用于预算匹配评分，附加房间列表用于生成书房、储物间、衣帽间等功能空间。

候选方案生成：推荐引擎首先构造面积变化因子集合和边界形态集合。面积变化因子包括紧凑型（0.85）、标准型（1.0）和舒适型（1.2）三种，用于缩放各房间标准尺寸；边界形态包括方正型和宽型两类，用于区分方案的整体空间倾向。系统对两组参数做笛卡尔积组合，形成初始候选空间。随后，`_build_room_list()` 按照固定优先级生成房间列表：客厅和厨房为核心必配空间；卧室根据用户期望卧室数生成，若卧室数不少于 2，则包含主卧和若干次卧；卫生间默认配置公共卫生间，并根据用户选择增加主卫；书房、餐厅、储物间等附加房间则根据用户需求逐一添加。各

房间尺寸由预设标准值乘以面积变化因子得到，从而形成不同尺度等级的候选方案。

邻接关系自动生成：候选方案生成后，系统根据房间列表自动推断邻接关系。基本规则包括：玄关与客厅邻接，并在存在走廊时与走廊形成 T 形邻接；走廊与客厅、卧室和公共卫生间建立普通邻接；若不存在走廊，则客厅与厨房直接邻接；客厅与餐厅、餐厅与厨房、主卧与主卫之间在同时存在时建立邻接关系。对于卧室、客厅和走廊等空间，系统还会构造扩展邻接关系，用于约束局部连通区域。与此同时，系统自动将客厅、卧室、书房和玄关等需要采光通风的房间加入靠外墙列表。

多维度评分体系：每个候选方案均从面积匹配、预算匹配、房间数量合理性和合规性四个维度计算匹配分数，总分为 100 分。其中，面积匹配权重为 30 分，根据用户需求面积中值与候选方案面积中值的相对偏差计算；预算匹配权重为 25 分，不同预算档次对应不同理想面积区间，候选面积越接近理想值，得分越高；合理性权重为 25 分，主要检查候选卧室数是否满足用户期望以及根据家庭人数推断出的最少卧室数；合规性权重为 20 分，采用扣分制检查核心房间是否缺失、房间总数是否过少、人均面积是否异常以及单间面积是否过大。家庭人数与最少卧室数的映射规则为：1 至 2 人至少 1 间卧室，3 人至少 2 间卧室，4 人及以上至少 3 间卧室。

表 5.1　智能推荐引擎评分维度

评分维度	权重	评分依据
面积匹配	30	用户需求面积中值与候选方案面积中值的相对偏差
预算匹配	25	经济、标准、豪华预算档对应的理想面积接近程度
合理性	25	卧室数量是否满足用户需求和家庭人数推断结果
合规性	20	核心房间、房间数量、人均面积和单间面积检查

推荐理由生成：系统自动为每个推荐方案生成匹配理由，例如“面积匹配度高”“包含独立书房”“预算档次匹配”等，并同步给出注意事项提示。

交互向导：3 步式向导对话框（基本需求 → 功能偏好 → 推荐结果），以卡片形式展示前 3 个推荐方案及其匹配分数、估算面积、预算区间和匹配理由。推荐引擎的需求输入界面和推荐结果界面分别如图 5.12 和 5.13所示。



图 5.12　户型智能推荐引擎界面



图 5.13　户型智能推荐结果界面

5.7 3D 建模集成实现

3D 建模模块（modules/3d_model.py，约 370 行）集成腾讯混元生 3D 大模型 API，界面效果如图 5.14所示。除自动根据户型结果生成提示词外，系统也支持手动输入描述生成 3D 模型，相关界面如图 5.15所示。系统从二维户型到三维模型的转换包括

两条路径：一是调用混元 3D API 进行云端生成，二是基于本地几何构造方法快速生成可预览的三维白模。

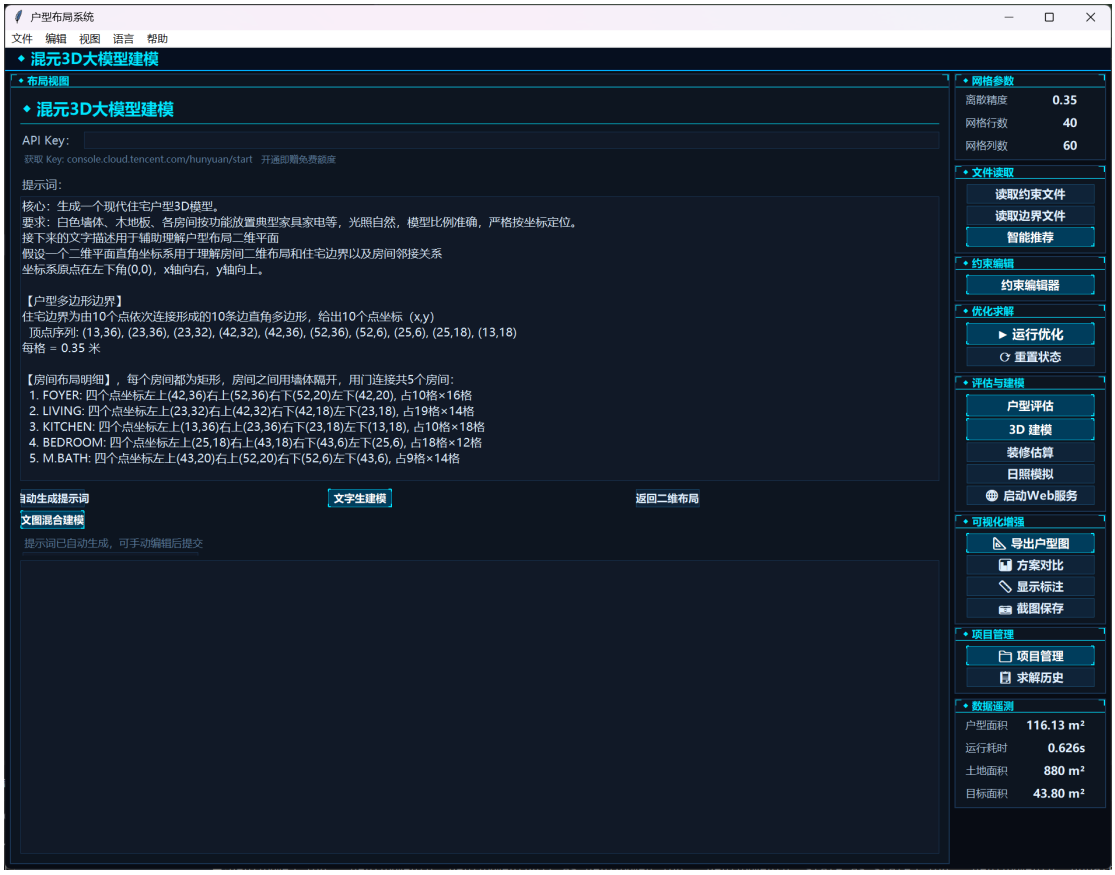


图 5.14　3D 建模功能界面

客户端封装：Hunyuan3DClient 类对 API 交互进行统一封装，包括认证（Authorization 头直接填写 sk-xxx，不需要添加 Bearer 前缀）、任务提交（POST /v1/ai3d/submit）、任务查询（POST /v1/ai3d/query，失败时自动重试）和结果轮询（最长等待 30 分钟，轮询间隔可配置）。

提示词构建：build_room_prompt 函数读取整数规划优化输出的布局数据，构造结构化文字描述，其中包含建筑多边形边界坐标、每个房间的四个角点坐标和网格尺寸，用于指导 3D 模型生成。

具体而言，二维优化结果由若干 RECTROOM 对象组成，每个对象包含房间标签、左上角坐标和宽高尺寸。由于界面绘图采用左上角为原点、y 轴向下的屏幕坐标系，而三维建模描述更适合使用左下角为原点、y 轴向上的数学坐标系，因此系统在构建提示词时执行坐标转换：$x' = x$，$y' = H_{grid} - y$。随后，系统为每个房间计算左

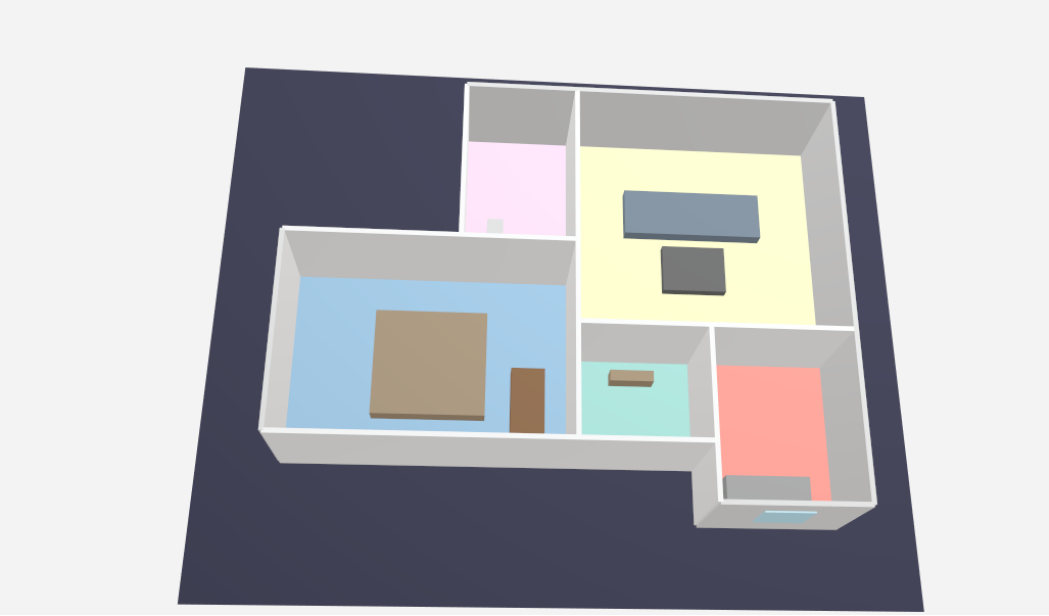


图 5.15　手动生成 3D 模型界面

上、右上、右下和左下四个角点，将其写入“房间布局明细”字段，并附加房间功能、网格尺寸和比例尺信息。当建筑边界中存在多边形顶点序列时，系统也会对边界顶点执行同样转换，并将其作为“户型多边形边界”写入提示词。

多模态支持：同时支持文生 3D（text_to_3d）、图生 3D（image_to_3d，支持本地文件/base64/URL 三种图片输入方式）和图文混合生成（image_with_prompt_to_3d，Sketch 模式）三种生成模式。

在实际测试中，单纯依赖文本描述生成三维模型时，模型对房间坐标、相对位置和邻接关系的理解存在不稳定性。为提高空间拓扑还原能力，系统默认采用图文混合模式：先由渲染器生成二维户型截图，再将截图编码为 base64 图像，同时提交结构化提示词。图像负责提供房间边界、整体轮廓和相对位置，文本负责补充房间名称、材质风格、家具需求和比例尺约束。该方式能够在一定程度上缓解纯文本空间推理不稳定的问题。

除云端生成路径外，系统还设计了本地三维模型生成逻辑。该方法以优化结果中的房间矩形、建筑边界和离散化比例为输入，依次构造地面、房间地板、墙体、门、窗和基础家具几何体，并通过 trimesh 导出 GLB 文件。墙体生成时，系统先提取每个房间的四条边，再判断该边是否为两个房间的共享边；若为共享边，则不生成墙体，避免相邻房间之间出现重复隔墙；若不是共享边，则根据该边是否贴近建筑边

界判断外墙或内墙，并采用不同颜色和厚度生成长方体墙体。门的位置由共享边决定，在共享边中点放置宽约 0.9m、高约 2.1m 的门洞示意；窗户仅在外墙边界上生成，并按照约 3m 间隔进行均匀分布。家具则根据房间标签从预设映射表中选择，例如卧室放置床和衣柜，客厅放置沙发和茶几，厨房放置橱柜，卫生间放置马桶和洗手台。该本地生成方法具有速度快、几何可控和离线可用的优点，但视觉真实感和材质细节不如云端大模型生成结果。

模型下载：get_download_urls 递归扫描响应 JSON 中所有以 http 开头的 URL 字段，download_model 方法支持下载模型文件到本地指定路径。生成的 3D 模型效果如图 5.16所示。



图 5.16　腾讯混元 3D 模型生成结果

5.8 本章小结

本章围绕系统详细设计与实现展开，分别介绍了约束编辑器、核心优化引擎、户型渲染器、日照采光模拟器、装修预算估算器、智能推荐引擎和 3D 建模集成模块的实现方式。约束编辑器负责将用户输入转化为规范化约束数据，核心优化引擎负责完成整数规划建模与求解，渲染器将求解结果转换为可读的户型平面图，日照、预算、推荐和 3D 模块则进一步扩展了系统的分析与展示能力。通过这些模块的协同工作，系统实现了从约束录入、优化生成到结果展示和辅助决策的完整流程。

6 系统测试与结果分析

6.1 测试环境与配置

系统测试在以下环境下进行：操作系统为 Windows 11 专业版，CPU 为 Intel Core i7 系列处理器，内存 16GB，Python 版本 3.10，Gurobi 优化求解器版本 11.0（学术许可），Tkinter 版本 8.6，Matplotlib 版本 3.7。

测试分为三个层次：单元测试用于验证各模块内部逻辑的正确性；集成测试用于验证模块之间的数据传递和接口协同；案例测试则基于真实户型需求运行端到端流程，以检验最终方案的合理性。

6.2 优化算法正确性验证

为验证整数规划优化算法的正确性，设计了一组 5 房间的标准测试用例，约束配置如表 6.1所示。

表 6.1　5 房间测试用例约束配置

房间名称	最小宽高比	最大宽高比	目标高度 (m)	目标宽度 (m)	类型
FOYER	1.0	2.0	1.5	1.5	玄关
LIVING	1.0	2.0	4.0	5.0	客厅
DINING	1.0	1.5	3.0	3.0	餐厅
KITCHEN	1.0	1.5	2.5	2.0	厨房
BATH	1.0	1.5	1.5	2.0	卫生间

约束条件包括：玄关与客厅邻接 (T 型)，玄关和客厅靠外墙。建筑边界为 22×20 格，离散化参数 $d = 0.35$ 米，对应实际尺寸约 $7.7m \times 7.0m$。

Gurobi 求解器在 2.3 秒内完成了优化求解，获得目标函数值 72.35。生成结果如表 6.2所示，系统运行结果界面如图 6.1所示。

验证结果表明：（1）所有 5 个房间互不重叠；（2）玄关与客厅共享边界（x=15 处），满足邻接约束；（3）玄关和客厅均贴靠建筑边界，满足靠外墙约束；（4）各房间的实际宽高比均落在用户指定的范围内。算法正确性得到了充分验证。

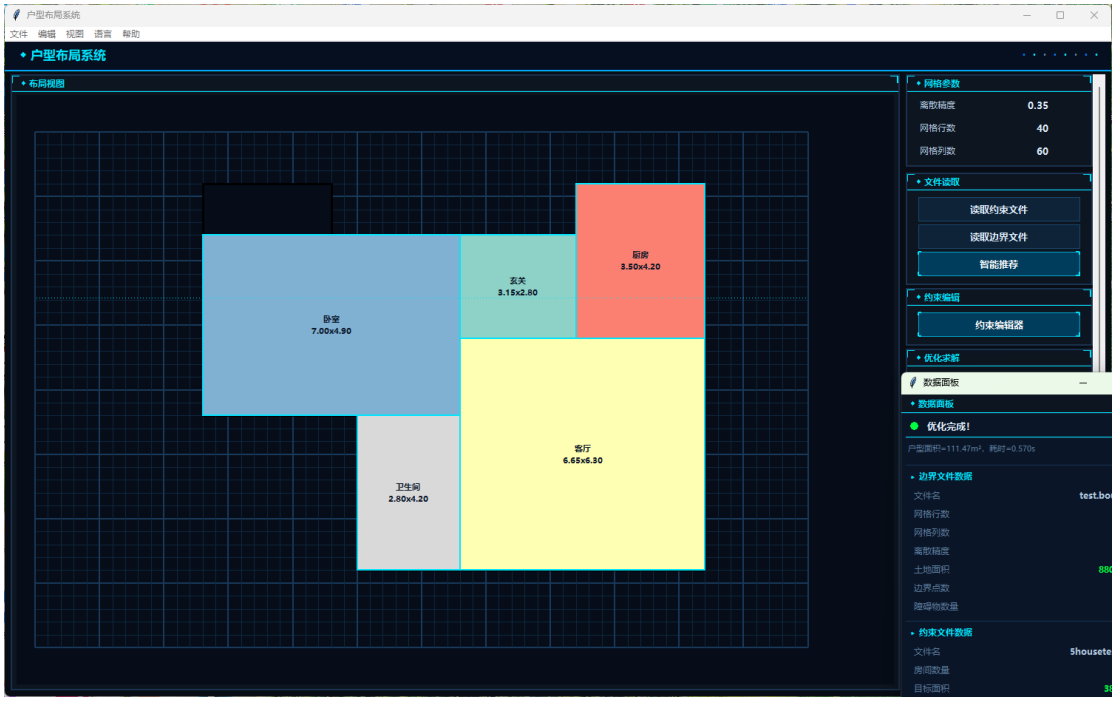


图 6.1　系统运行结果界面

表 6.2　5 房间测试用例生成结果

房间	坐标 (x,y)	宽度 (格)	高度 (格)	实际尺寸 (m)	面积 (m²)
FOYER	(15, 5)	4	3	1.40×1.05	1.47
LIVING	(0, 8)	13	12	4.55×4.20	19.11
DINING	(0, 0)	8	8	2.80×2.80	7.84
KITCHEN	(8, 0)	6	8	2.10×2.80	5.88
BATH	(14, 0)	5	5	1.75×1.75	3.06

6.3 多房间规模性能测试

为评估算法在不同规模下的求解表现，本文设计了 8 房间、12 房间和 15 房间三组递增规模的测试用例，并记录从建模到求解完成的总耗时和目标值。结果见表 6.3。

表 6.3　不同房间数量下的求解性能对比

房间数量	二元变量数	约束数量	求解时间 (s)	目标值
5	约 50	约 180	2.3	72.4
8	约 120	约 450	8.7	156.8
12	约 260	约 980	25.4	312.3
15	约 400	约 1520	42.1	487.6

从测试结果可以看出，求解时间随房间数量的增加呈超线性增长（从 2.3 秒到 42.1 秒），这与整数规划问题的 NP-hard 性质一致。但即便对于 15 个房间的中等规模问题，求解仍在 1 分钟以内完成，完全满足实际使用需求。对于更大规模（如购物中心 >100 房间）的场景，可放宽最优性要求或启用在较短时间内返回当前最佳可行解。

6.4 户型评估功能验证

使用系统内置的 FloorplanEvaluator 对上述 5 房间方案进行综合评估，评估结果如下：

- 综合评分：86.2/100，等级 B（良好）
- 空间利用率：87.5/100 —有效面积比为 82.3%，走廊面积占比 9.1%，均处于理想区间
- 宽高比合规：93.8/100 —所有 5 个房间的宽高比均在约束范围内
- 动线效率：85.0/100 —玄关 → 客厅 1 步（理想）、客厅 → 厨房 1 步（理想）、卧室 → 卫生间不可达（缺少卧室导致扣分）
- 自然采光：80.0/100 —客厅和玄关靠外墙，重要房间采光率 100%
- 动静分区：78.3/100 —私密区聚类度良好，公私区有一定分离
- 邻接满意度：90.0/100 —所有设计的邻接关系均被实际共享边界满足
- 风水参考分：70/100 —大门不对卫生间（+25），厨房不在中心（+15），卫生间不对床（+15），客厅双向靠墙（+15）

评估报告界面如图 6.2所示。



图 6.2　户型评估报告界面

6.5 日照模拟功能验证

以合肥（纬度 31.86°）为例，设定朝向为南向、日期为 2026 年 6 月 21 日（夏至），对 5 房间方案运行日照模拟。模拟输出各房间日照时数分别为：客厅 7.2h、玄

关 5.8h、厨房 3.1h、餐厅 2.4h、卫生间 1.8h。客厅和玄关因靠南向外墙且窗户面积较大，获得了充足的日照；卫生间位于北侧内部，日照较少，结果符合建筑采光的基本物理规律。

6.6 预算估算功能验证

对 5 房间方案（总面积约 37.36 平方米）分别进行经济简装、标准精装和高端豪装三个档次的预算估算。结果分别为：经济简装约 37,360 元、标准精装约 74,720 元、高端豪装约 149,440 元。系统自动生成了九项工程类别的分项预算（如地面工程占 18

系统功能完整性测试

对所有功能模块进行了端到端的集成测试，测试结果汇总如下：

1. 约束编辑器：支持房间添加/删除/编辑/排序，邻接关系可视化创建和删除，约束文件导入/导出（.cons 格式），约束合法性校验。全部通过。
2. 优化引擎：支持多房间（5-15 间）户型生成，支持一层/两层住宅模式，支持更新模式（Preserve/Merge）。全部通过。
3. 户型渲染：支持 PNG/SVG/PDF/DXF 导出，支持尺寸标注、门窗标记、家具示意、图例指北针。全部通过。
4. 日照模拟：支持 8 个朝向、多日期选择、热力图和柱状图生成、文本报告导出。全部通过。
5. 预算估算：支持 3 个装修档次、分项预算和房间明细、饼图和柱状图生成、文本报告导出。全部通过。
6. 智能推荐：支持 3 步向导式交互、多维度匹配评分、推荐理由生成。全部通过。
7. 3D 建模：支持文生 3D、图生 3D 和图文混合三种模式的任务提交，支持轮询和结果下载。全部通过（需有效的 API Key）。

测试过程中未发现导致系统崩溃或输出错误的严重缺陷，系统功能完整性和稳定性达到了毕业设计的预期目标。

6.7 本章小结

本章对系统进行了测试与结果分析。首先说明了测试环境和测试方法，然后通过 5 房间标准用例验证了优化算法在不重叠、邻接、靠外墙和宽高比等约束方面的

正确性；接着对不同房间数量下的求解性能进行了对比，结果表明系统在典型住宅规模下能够在可接受时间内完成求解；随后分别验证了户型评估、日照模拟、预算估算和各功能模块的完整性。测试结果说明，系统能够稳定完成住宅户型自动生成及相关辅助分析任务，基本达到毕业设计预期目标。

7 总结与展望

7.1 工作总结

本文完成的主要工作可归纳如下：

（1）将整数规划方法应用于住宅户型生成任务。本文将尺寸、不重叠、邻接、边界、扩展邻接和靠墙等七类建筑约束逐条转化为数学约束，并将空间利用率和尺寸精度等设计目标组织为带权目标函数，最终形成完整的整数规划模型并交由 Gurobi 求解。求解参数经过针对性配置，其中 MIPFocus=2 用于在可接受时间内优先搜索高质量解，Heuristics=0.1 用于控制启发式搜索强度。

（2）实现了完整的软件系统。系统由约束编辑器、优化引擎、渲染器、日照模拟、预算估算、推荐引擎和 3D 建模等模块组成，采用 Python 和 Tkinter 实现，并通过分层架构降低模块间耦合。

（3）实现了较为完整的户型渲染功能。系统生成的平面图包含房间色块、尺寸标注、门窗标识、家具示意、图例和指北针等信息，并支持 PNG、SVG、PDF 和 DXF 等格式导出，便于屏幕查看、打印和专业软件后续处理。

（4）构建了户型评分体系。系统从空间利用率、宽高比、动线效率、自然采光、动静分区和邻接满意度六个维度对方案进行评分，为用户评估方案优劣提供量化参考。

（5）集成了多类辅助工具。日照模拟模块基于天文学公式计算各房间采光情况，预算估算模块根据装修档次计算成本，推荐引擎则帮助用户从模板库中快速匹配合适户型。

7.2 系统存在的不足

系统仍存在以下不足：

（1）建筑边界目前主要支持矩形。对于 L 形、U 形等非矩形轮廓，系统尚不能完整处理，这在实际项目应用中具有一定限制。虽然算法数据结构中已经预留了 boundingPoint 和 obstacle 字段，但主流程尚未完整接入非凸多边形约束建模。

（2）目标尺寸仍依赖用户手动填写。各房间宽度和高度主要由用户经验决定。Wu 等人在 2018 年的研究中提出可基于真实住宅平面图训练多项式回归模型以自动生成约束，该思路具有进一步研究价值。

（3）目标函数的三个权重（areaWeight、errorWeight、extraWeight）目前为固定配置。若后续能够根据用户偏好或户型特征自动调整权重，生成结果有望进一步提升。

（4）渲染器中的家具摆放主要基于模板生成，尚未充分考虑房间实际形状和尺寸是否满足家具布置要求。日照模拟中的窗户位置和大小也主要根据经验估算，而非来源于真实窗洞设计，因此采光模拟结果与实际情况可能存在偏差。

（5）国际化与 Web 端功能仍不完善。当前多语言界面仅完成基础框架，Web 服务器（web_server.py）也仅包含基础 HTTP 服务骨架，距离完整浏览器端交互仍有较大差距。

7.3 未来工作展望

针对上述不足，后续可从以下方向继续完善：

（1）支持非矩形建筑边界。可考虑采用多边形凸分解方法，例如将 L 形边界拆分为多个矩形子区域分别求解，并结合边界点序列遍历和多段接触约束建模，将复杂建筑轮廓纳入整数规划框架。

（2）采用数据驱动方式自动生成约束。后续可收集真实住宅户型数据，训练回归模型或神经网络，使系统能够根据总面积和房间数量自动预测各房间合理尺寸和邻接关系，从而减少用户手动输入成本。

（3）开展多目标优化与交互式探索。当前目标函数采用加权求和方式，本质上仍属于单目标优化。若改为 Pareto 多目标框架，用户可在面积利用率和尺寸精确度之间进行灵活权衡；同时，可进一步设计交互式方案探索界面，使用户调整约束后能够快速查看布局变化。

（4）加强 3D 可视化与 VR 体验。当前系统已具备 2D 渲染和云端 3D 模型生成能力，但尚缺少本地实时 3D 渲染视图。后续可基于 OpenGL 或 WebGL 实现可交互三维漫游，使用户能够在虚拟空间中体验生成户型。

（5）完善 Web 端功能。后续可继续实现 web_server.py 中的完整业务逻辑，将系统扩展为 Web 应用，使用户能够在浏览器中完成户型设计、优化生成和结果查看的完整流程。

总体而言，本文系统在算法建模与工程实现方面形成了较为完整的方案，为住宅户型自动化设计提供了可运行的软件原型。随着后续功能完善和性能优化，该系统具备进一步拓展到实际辅助设计场景的潜力。

7.4 本章小结

本章对全文工作进行了总结，并对系统存在的问题和未来改进方向进行了说明。本文完成了基于整数规划的住宅户型自动生成系统设计与实现，形成了从约束建模、优化求解、平面图渲染到日照分析、预算估算、智能推荐和 3D 建模的完整流程。同时，系统仍存在非矩形边界支持不足、约束参数依赖人工输入、目标函数权重固定、家具与窗户建模较粗略等问题。未来可从复杂边界建模、数据驱动约束生成、多目标优化、实时 3D 可视化和 Web 端完善等方向继续改进。

参考文献

[1] 彭莉妮. 装配式住宅户型模块标准化设计要点分析[J]. 住宅与房地产, 2021. (25): 124-125.

[2] 杨艳丽. 探析板式高层住宅户型的可变性设计要点[J]. 建材与装饰, 2020. (6): 96-97.

[3] Alan Anderson, Andrew Marsters, Brian Dossick, Guy Neff. BIM and Education: Current Trends in Architectural Education and the Impact of Building Information Modeling[M]. McGraw-Hill Construction, 2011.

[4] J. Liu, Y. Xue, H. Ni, et al. Computer-Aided Layout Generation for Building Design: A Review [J]. Computational Visual Media, 2025.

[5] Forest Flager, John Haymaker. A Comparison of Multidisciplinary Design, Analysis and Optimization Processes in the Building Construction and Aerospace Industries[C]//24th International Conference on Information Technology in Construction. Maribor, Slovenia, 2007: 625-630.

[6] George Stiny, James Gips. Shape Grammars and the Generative Specification of Painting and Sculpture[C]//IFIP Congress: vol. 2: 3. 1971: 125-135.

[7] Yoav I. H. Parish, Pascal Müller. Procedural Modeling of Cities[C]//Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH). 2001: 301-308.

[8] Pascal Müller, Peter Wonka, Simon Haegler, Andreas Ulmer, Luc Van Gool. Procedural Modeling of Buildings[J]. ACM Transactions on Graphics (TOG), 2006, 25(3): 614-623.

[9] Z. Li, S. Li, G. Hinchcliffe, et al. Automated Architectural Space Layout Planning Using a Physics-Inspired Generative Design Framework[J]. arXiv preprint arXiv:2406.14840, 2024.

[10] 李若天. 基于遗传算法的适应性住宅套内平面布局生成设计研究[J]. 城市建筑, 2024, 21(15): 148-154+167.

[11] 李冰瑶. 基于多性能目标优化的住宅规划布局设计方法研究[D]. 深圳大学, 2018.

[12] Graziella Laignel, Nicolas Pozin, Xavier Geffrier, Loukas Delevaux, Florian Brun, Bastien Dolla. Floor Plan Generation Through a Mixed Constraint Programming-Genetic Optimization Approach [J]. Automation in Construction, 2021, 123: 103491.

[13] Nelson Nauata, Kai-Hung Chang, Chin-Yi Cheng, Greg Mori, Yasutaka Furukawa. House-GAN: Relational Generative Adversarial Networks for Graph-Constrained House Layout Generation[C] //Proceedings of the European Conference on Computer Vision (ECCV). 2020: 162-177.

[14] Ruizhen Hu, Zeyu Huang, Yuhan Tang, Oliver van Kaick, Hao Zhang, Hui Huang. Graph2Plan: Learning Floorplan Generation from Layout Graphs[J]. ACM Transactions on Graphics (TOG), 2020, 39(4): 118:1-118:14.

[15] Chen Liu, Jiajun Wu, Pushmeet Kohli, Yasutaka Furukawa. Raster-to-Vector: Revisiting Floorplan Transformation[C]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017: 2195-2203.

[16] David L. Donoho. Compressed Sensing[J]. IEEE Transactions on Information Theory, 2006, 52(4): 1289-1306.

[17] 陈圣格, 周婷, 陈志华, 等. 模块建筑参数化平面优化及智能化结构设计方法[J]. 重庆大学学报, 2021, 44(9): 51-66.

[18] Wenming Wu, Lubin Fan, Ligang Liu, Peter Wonka. MIQP-based Layout Design for Building Interiors[J]. Computer Graphics Forum (Eurographics 2018), 2018, 37(2): 511-521.

[19] Anna Haratyunyan. Generative Design for Apartment Layout: A Constraint-Based Approach[J]. Nexus Network Journal, 2019, 21(2): 303-321.

[20] Christos H. Papadimitriou, Kenneth Steiglitz. Combinatorial Optimization: Algorithms and Complexity[M]. Courier Corporation, 1998.

[21] George L. Nemhauser, Laurence A. Wolsey. Integer and Combinatorial Optimization[M]. New York: John Wiley & Sons, 1988.

[22] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual[M]. Houston, TX, USA: Gurobi Optimization, LLC, 2024.

[23] P. I. Cooper. The Absorption of Radiation in Solar Stills[J]. Solar Energy, 1969, 12(3): 333-346.

致谢

大学四年的学习生活即将结束，回顾这一阶段的成长经历，收获颇多，也深感不易。

首先要感谢我的指导老师吴文明老师。从选题确定到方案设计，从系统实现到论文撰写，吴老师始终给予我耐心指导和及时帮助。在算法设计、Gurobi 参数调试和界面实现遇到困难时，老师的建议常常能够帮助我理清思路。老师严谨负责的治学态度和宽厚真诚的为人处世方式，使我受益良多。

感谢合肥工业大学计算机与信息学院四年来的培养。学院提供的学习环境、实验条件和课程体系为本文工作奠定了重要基础。也感谢每一位授课老师，是各位老师的教学和引导使我逐步建立起计算机专业知识体系。

感谢实验室的同学们在学习和科研过程中的陪伴与帮助。与大家的讨论使我解决了许多实现过程中的问题，也让我在系统开发和论文写作过程中获得了持续的支持。

感谢 Gurobi Optimization 公司提供的学术免费许可，也感谢腾讯混元团队提供的 3D 大模型 API 支持。这些工具为系统优化求解和三维建模功能的实现提供了重要保障。

最后，感谢我的家人长期以来的理解、支持和鼓励。无论是在学习顺利时，还是在项目开发遇到困难时，家人的支持始终是我坚持完成学业和毕业设计的重要力量。

谨以此文向所有帮助和关心过我的老师、同学、朋友和家人表示衷心感谢。

作者：王景涛

2026 年 5 月 15 日

附录

本附录对系统实现中与户型生成、整数规划建模及三维模型构建相关的核心数据结构和主要算法流程进行归纳说明。为突出实现逻辑，以下代码在不影响语义的前提下进行了适当简化，省略了界面交互、异常处理和部分重复约束构造细节。

1. 核心数据结构定义

系统首先将建筑边界、房间对象和约束集合统一抽象为结构化数据。其中，矩形房间采用离散网格坐标表示，建筑边界同时保存包围盒、障碍物、多边形顶点和占据图等信息，为后续整数规划建模提供基础输入。

```
@dataclass
class RECTROOM:
    index: int = 0
    label: str = ""
    x: int = 0
    y: int = 0
    width: int = 0
    height: int = 0

@dataclass
class BOUNDARY:
    x: int = 0
    y: int = 0
    width: int = 0
    height: int = 0
    obstacle: List[RECTROOM] = field(default_factory=list)
    boundingPoint: List[Tuple[int, int]] = field(default_factory=list)
    boundingMap: List[List[int]] = field(default_factory=list)
    landArea: int = 0
```

房间尺寸约束由输入文件中的浮点参数转换而来，主要包括最小尺寸、目标尺寸和宽高比范围。完整约束集进一步组织邻接关系、扩展邻接关系、贴边房间和贴墙房间等信息。优化变量则对应每个房间的位置、尺寸和朝向选择。

```
@dataclass
class FLOATDIMENSION:
    label: str = ""
    heightMin: float = 0.0
    widthMin: float = 0.0
    minRatio: float = 0.0
    maxRatio: float = 0.0
    heightObj: float = 0.0
    widthObj: float = 0.0
```

```
@dataclass
class CONSTRAINT:
    floatDimension: List[FLOATDIMENSION] = field(default_factory=list)
    roomAdjacency: List[ADJACENCY] = field(default_factory=list)
    extentedAdjacency: List[EXADJACENCY] = field(default_factory=list)
    roomBoundary: List[str] = field(default_factory=list)
    roomWall: ROOMWALL = field(default_factory=ROOMWALL)

@dataclass
class VARIABLE:
    var0 = None         # y coordinate
    var1 = None         # x coordinate
    var2 = None         # height
    var3 = None         # width
    orientation = None # binary orientation variable
```

2. 约束文件与边界文件解析

系统通过 read_constraint_file() 解析 .cons 文件。该文件按功能分为尺寸约束、邻接约束、扩展邻接约束、贴边约束和贴墙约束等段落。解析过程逐行读取文件内容，根据段落标识构造相应数据对象，并写入 CONSTRAINT 结构。

```
def read_constraint_file(filepath: str) -> CONSTRAINT:
    constraint = CONSTRAINT()
    lines = read_all_lines(filepath)
    idx = 0

    if lines[idx] == "SUBDIVISION":
        constraint.genrationType = "SUBDIVISION"
        idx += 1

    if lines[idx] == "DIMENSION":
        idx += 1
        while idx < len(lines) and lines[idx] != "":
            parts = lines[idx].split()
            room = FLOATDIMENSION()
            room.label = parts[0]
            room.minRatio = float(parts[1])
            room.maxRatio = float(parts[2])
            room.heightObj = float(parts[3])
            room.widthObj = float(parts[4])
            room.heightMin = room.heightObj * 0.5
            room.widthMin = room.widthObj * 0.5
            constraint.floatDimension.append(room)
            idx += 1

    if lines[idx] == "ADJACENCY":
```

```
        idx += 1
        while idx < len(lines) and lines[idx] != "":
            label1, label2 = lines[idx].split()[:2]
            constraint.roomAdjacency.append(
                ADJACENCY(label1=label1, label2=label2))
            idx += 1

    return constraint
```

建筑边界文件由 read_boundary_file() 解析。系统读取多边形顶点后，先计算边界包围盒，再通过射线法生成占据图，最后将不可用区域转换为矩形障碍物，用于后续非重叠约束和贴边约束建模。

```
def read_boundary_file(filepath: str) -> BOUNDARY:
    row, col, discretization, point_count = read_header(filepath)
    points = read_boundary_points(filepath, point_count)

    boundary = bounding_point_to_boundary(points)
    boundary.boundingMap = fill_polygon_by_ray_casting(points)
    boundary.obstacle = Area2Rectangle(boundary.boundingMap)
    boundary.landArea = count_available_cells(boundary.boundingMap)
    return boundary
```

3. 整数规划模型构建

在优化模型中，每个房间对应四个整数变量和一个二进制朝向变量。整数变量分别表示房间左上角网格坐标及其宽高，朝向变量用于描述房间横向或纵向布置。模型求解器采用 Gurobi，变量上下界由建筑包围盒和最小房间尺寸共同确定。

```
def CreatVariable(self):
    minScale = max(1, math.ceil(ROOMMINSIZE * self.g_areaRatio))
    bb = self.boundingBox

    for i in range(self.roomNumber):
        room = VARIABLE()
        room.var0 = self.model.addVar(lb=bb.y, ub=bb.y + bb.height,
                                      vtype=GRB.INTEGER, name=f"y_{i}")
        room.var1 = self.model.addVar(lb=bb.x, ub=bb.x + bb.width,
                                      vtype=GRB.INTEGER, name=f"x_{i}")
        room.var2 = self.model.addVar(lb=minScale, ub=bb.height,
                                      vtype=GRB.INTEGER, name=f"h_{i}")
        room.var3 = self.model.addVar(lb=minScale, ub=bb.width,
                                      vtype=GRB.INTEGER, name=f"w_{i}")
        room.orientation = self.model.addVar(vtype=GRB.BINARY,
                                             name=f"ori_{i}")
        self.houseVar.append(room)
```

尺寸约束用于限制房间宽高、宽高比和边界范围。系统利用 Big-M 方法根据朝向变量切换不同的宽高比约束，从而在同一模型中支持横向和纵向两类布置方式。

```
def DimensionConstraint(self):
    bb = self.boundingBox
    for i in range(self.roomNumber):
        dim = self.intDimension[i]
        hv = self.houseVar[i]

        self.model.addConstr(
            hv.var2 >= dim.heightMin +
            (dim.widthMin - dim.heightMin) * hv.orientation)
        self.model.addConstr(
            hv.var3 >= dim.heightMin +
            (dim.widthMin - dim.heightMin) * (1 - hv.orientation))

        self.model.addConstr(dim.minRatio * hv.var2 <=
                             hv.var3 + LARGE * hv.orientation)
        self.model.addConstr(dim.maxRatio * hv.var2 >=
                             hv.var3 - LARGE * hv.orientation)
        self.model.addConstr(dim.minRatio * hv.var3 <=
                             hv.var2 + LARGE * (1 - hv.orientation))
        self.model.addConstr(dim.maxRatio * hv.var3 >=
                             hv.var2 - LARGE * (1 - hv.orientation))

        self.model.addConstr(hv.var0 + hv.var2 <= bb.y + bb.height)
        self.model.addConstr(hv.var1 + hv.var3 <= bb.x + bb.width)
```

不重叠约束通过四个二进制分离变量表达两个矩形房间的相对位置关系。对任意两个房间，至少需要满足“上、下、左、右”中的一个分离方向。房间与障碍物之间采用相同思想处理。

```
def NonOverlapConstraint(self):
    for i in range(self.roomNumber):
        for j in range(i + 1, self.roomNumber):
            hv_i = self.houseVar[i]
            hv_j = self.houseVar[j]
            b0 = self.model.addVar(vtype=GRB.BINARY)
            b1 = self.model.addVar(vtype=GRB.BINARY)
            b2 = self.model.addVar(vtype=GRB.BINARY)
            b3 = self.model.addVar(vtype=GRB.BINARY)

            self.model.addConstr(hv_j.var0 + hv_j.var2 <=
                                 hv_i.var0 + LARGE * (1 - b0))
            self.model.addConstr(hv_j.var0 >=
                                 hv_i.var0 + hv_i.var2 - LARGE * (1 - b1))
            self.model.addConstr(hv_j.var1 + hv_j.var3 <=
                                 hv_i.var1 + LARGE * (1 - b2))
            self.model.addConstr(hv_j.var1 >=
                                 hv_i.var1 + hv_i.var3 - LARGE * (1 - b3))
```

```
            self.model.addConstr(b0 + b1 + b2 + b3 >= 1)
```

邻接约束用于表达两个房间需要共享一定长度边界的设计要求。系统分别支持普通邻接、T 形邻接和扩展邻接。普通邻接中，二进制变量用于选择水平或垂直接触方向，并保证两个房间在另一方向上的投影重叠长度不小于给定阈值。

```
def TobeAdjacency(self, room1, room2):
    commonLength = max(1, Round(ROOMTOCHSIZE * self.g_areaRatio))
    touch = self.model.addVar(vtype=GRB.BINARY)

    self.model.addConstr(room1.var0 + room1.var2 >=
                         room2.var0 + commonLength * touch)
    self.model.addConstr(room1.var0 <=
                         room2.var0 + room2.var2 - commonLength * touch)
    self.model.addConstr(room1.var1 + room1.var3 >=
                         room2.var1 + commonLength * (1 - touch))
    self.model.addConstr(room1.var1 <=
                         room2.var1 + room2.var3 - commonLength * (1 - touch))
```

贴边与贴墙约束用于保证指定房间与建筑外边界或障碍物边界相接触。模型为不同边界方向建立二进制变量，并通过接触变量之和控制房间至少或恰好贴合某一边界。

```
def TobeRoomBoundary(self, room):
    commonLength = max(1, Round(ROOMTOCHSIZE * self.g_areaRatio))
    bb = self.boundingBox

    t0 = self.model.addVar(vtype=GRB.BINARY)
    t1 = self.model.addVar(vtype=GRB.BINARY)
    t2 = self.model.addVar(vtype=GRB.BINARY)
    t3 = self.model.addVar(vtype=GRB.BINARY)

    self.model.addConstr(room.var0 <= bb.y + LARGE * (1 - t0))
    self.model.addConstr(room.var0 + room.var2 >=
                         bb.y + bb.height - LARGE * (1 - t1))
    self.model.addConstr(room.var1 <= bb.x + LARGE * (1 - t2))
    self.model.addConstr(room.var1 + room.var3 >=
                         bb.x + bb.width - LARGE * (1 - t3))

    touch_sum = t0 + t1 + t2 + t3
    for obs in self.obstacle:
        touch_i = self.model.addVar(vtype=GRB.BINARY)
        touch_sum += touch_i
    self.model.addConstr(touch_sum == 1)
```

目标函数由三部分组成：未分配面积、尺寸偏差和辅助空间面积。未分配面积用于提高土地利用率，尺寸偏差用于使生成户型接近期望尺寸，走廊和楼梯等辅助

空间惩罚项用于避免无效面积过大。

```
def SetObjection(self):
    bb = self.boundingBox
    area = bb.width * bb.height
    for obs in self.obstacle:
        area -= obs.width * obs.height
    for i in range(self.roomNumber):
        area -= self.houseVar[i].var2 * self.houseVar[i].var3

    error = 0
    for i in range(self.roomNumber):
        dim = self.intDimension[i]
        hv = self.houseVar[i]
        term1 = hv.var2 - hv.orientation * dim.widthObj \
                - (1 - hv.orientation) * dim.heightObj
        term2 = hv.var3 - (1 - hv.orientation) * dim.widthObj \
                - hv.orientation * dim.heightObj
        error += term1 * term1 + term2 * term2

    extraArea = 0
    for i in range(self.roomNumber):
        label = self.intDimension[i].label.upper()
        if "CORRIDOR" in label or "STAIRS" in label:
            extraArea += self.houseVar[i].var2 * self.houseVar[i].var3

    self.model.setObjective(self.areaWeight * area +
                            self.errorWeight * error +
                            self.extraWeight * extraArea,
                            GRB.MINIMIZE)
```

4. 求解流程与自适应重试机制

系统的户型生成流程由 `Tiling()` 统一调度。该函数依次完成变量创建、尺寸约束、不重叠约束、邻接约束、贴边贴墙约束、附加约束和目标函数构造，然后调用 Gurobi 求解器进行优化。求解完成后，系统从变量值中提取矩形房间结果，并转换为可视化和后续三维建模所需的数据格式。

```
def Tiling(self):
    self.Initialization()
    self.CreatVariable()
    self.DimensionConstraint()
    self.NonOverlapConstraint()
    self.AdjacencyConstraint()
    self.ExAdjacencyConstraint()
    self.RoomWallConstraint()
    self.RoomBoundaryConstraint()
    self.OtherConstraint()
```

```
    self.SetObjection()

    self.model.optimize()
    self.model_status = self.model.Status

    for i in range(self.roomNumber):
        rectRoom = RECTROOM()
        rectRoom.index = i
        rectRoom.label = self.intDimension[i].label
        rectRoom.x = Round(self.houseVar[i].var1.X)
        rectRoom.y = Round(self.houseVar[i].var0.X)
        rectRoom.width = Round(self.houseVar[i].var3.X)
        rectRoom.height = Round(self.houseVar[i].var2.X)
        self.blockRoom.append(rectRoom)
```

在求解器初始化阶段，系统设置启发式搜索、优化重点、预求解策略、割平面和超时时间等参数，以平衡求解质量和运行时间。

```
def Initialization(self):
    self.model.setParam(GRB.Param.Heuristics, 0.1)
    self.model.setParam(GRB.Param.MIPFocus, 2)
    self.model.setParam(GRB.Param.Presolve, 2)
    self.model.setParam(GRB.Param.Cuts, 0)
    self.model.setParam(GRB.Param.TimeLimit, 500)
```

考虑到用户输入约束可能过紧，GUI 层设计了自适应重试机制。当模型无可行解时，系统会逐步缩小房间最小尺寸并重新构建模型，最多重试指定次数，从而提高实际使用中的求解成功率。

```
false_times = 0
max_retries = 20

while false_times <= max_retries:
    constraint = CONSTRAINT()
    constraint.floatDimension = working_dimensions

    area_ratio = math.sqrt(self.land_area / self.target_area)
    fp = Floorplan(storey_boundary=self.boundary,
                   storey_constraint=constraint,
                   state=STATE.GenerateGeneral,
                   area_ratio=area_ratio)
    fp.run()

    if fp.model.SolCount > 0 and len(fp.blockRoom) == len(working_dimensions):
        self.block_room = fp.blockRoom
        break

    false_times += 1
    for dim in working_dimensions:
```

```
        dim.heightMin *= 0.9
        dim.widthMin *= 0.9
```

5. 本地三维模型生成流程

除调用外部三维生成接口外，系统还提供本地三维模型生成模块。该模块基于 trimesh 将二维户型数据转换为三维场景，主要包括地面、房间地板、墙体、门、窗和基础家具等几何对象，最终导出为 .glb 文件。

```
class Local3DGenerator:
    def __init__(self):
        self.wall_height = 2.8
        self.wall_thickness = 0.12

    def generate(self, block_rooms, boundary, discretization, output_path):
        d = discretization
        meshes = []

        if boundary is not None:
            meshes.append(self._make_ground(boundary, d))

        for room in block_rooms:
            meshes.append(self._make_floor(room, d))

        meshes.extend(self._make_walls(block_rooms, boundary, d))
        meshes.extend(self._make_doors(block_rooms, d))
        meshes.extend(self._make_windows(block_rooms, boundary, d))
        meshes.extend(self._make_furniture(block_rooms, d))

        scene = trimesh.Scene()
        for mesh in meshes:
            scene.add_geometry(mesh)
        scene.export(file_obj=output_path, file_type="glb")
        return output_path
```

墙体生成的关键在于识别房间边界与共享边。系统遍历每个房间的四条边，若某条边为两个房间共享边，则不生成墙体；否则根据其是否位于建筑外边界区分内墙和外墙，并构造相应的长方体网格。

```
def _make_walls(self, rooms, boundary, d):
    meshes = []
    all_edges = []
    for room in rooms:
        for edge in extract_edges(room):
            if not self._shared(edge, rooms):
                all_edges.append(edge)
```

```
    for edge in all_edges:
        external = self._on_boundary(edge, boundary)
        color = ext_color if external else int_color
        meshes.append(make_wall_box(edge, self.wall_thickness,
                                    self.wall_height, color))
    return meshes

def _make_doors(self, rooms, d):
    shared_edges = collect_shared_edges(rooms)
    return make_door_boxes(shared_edges, width=0.9, height=2.1)

def _make_windows(self, rooms, boundary, d):
    external_edges = collect_external_edges(rooms, boundary)
    return make_window_boxes(external_edges, interval=3.0, sill_height=1.0)
```

通过上述流程，系统能够将整数规划求解得到的二维矩形户型进一步转换为可浏览、可导出的三维模型，为户型方案展示和后续空间分析提供支持。